

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_0febe894-ff9\agent-  
a1e29fa143770e786.jsonl`  
- SHA-256: `4ee8cc27ade62c574aaa76e9523b37ba7ba905ebb50ae4e21bbf8047c87690f2`  
- Source modified: `2026-07-07T03:39:30+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_0febe894-ff9`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-07T03:35:16.489Z)

Reviewing GitHub PR #58 "docs: refresh current project documentation" (branch agent/docs-sweep
-> main). DOCS-ONLY refresh + a few
doc-accuracy source edits (help text, docstrings, Dockerfile comment, pyproject description).
+782/-787, 14 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr58.
This is an ACCURACY review: the docs must match the MERGED code (Tracks B-F + ADR 0012 F1-F7 are
all merged). Read the doc diff:

```
git -C C:/Users/avidu/Projects/Telegram diff 7fadffb..origin/agent/docs-sweep -- <file>  
and cross-check every concrete claim against the actual code (src/tg_video_filter/*.py,  
docs/adr/*). Read code via Read/Grep; run  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check defaults/behavior.
```

KEY GROUND TRUTH (verified by the main reviewer) to check the docs against:

- Selective scanning is OPT-IN: new sources default watched=false (should_ingest_source); the
watched flag is user-controlled (set_source_watched / /watch,/unwatch);
upsert_source no longer clobbers watched (a fix in #41). Commands: /channels, /watch
<id|@name>, /unwatch, /watchlist.
- FilterConfig exclusion DEFAULTS: exclude_gif=True, exclude_round=True, exclude_nosound=False
(i.e. GIF/round excluded, nosound allowed). The old "all off by default" was WRONG.
- Policy pipeline stages (evaluate_pipeline order): topic-preferences -> url-reputation ->
caption-safety -> deep-inspection. Deep inspection is a budget BOUNDARY only; NO production
fetcher is wired (ctx.fetcher/budget are None in the live path), so enabling deep_inspect drops
items.
- Config lives in policies.config_json (raw dict); save_filter_config MERGES flat FilterConfig
keys so ADR 0012 keys (topics/security_checks/deep_inspect) survive a /set. There is NO /set
command to set those policy keys yet (dormant).
- Tables: accounts, telegram_sessions, sources, content_items, policies, destinations,
routing_rules, delivery_attempts, policy_decisions (+ *_deprecated). policy_decisions has NO
TTL/retention cleanup (deferred, issue #56).
- Dedup: 3-tier (message instance, Telegram document.id, metadata fingerprint) via
content_items.dedup_key. Delivery modes forward/link/both to me/@username/chat id.
- /why <content_item_id> is account-scoped (get_content_item_for_account), renders the stored
policy trace or a caveated metadata replay; markdown-escaped.

Ground rules: READ the doc diff AND the code, cite file:line. Report ONLY doc claims in THIS PR
diff that are FACTUALLY WRONG or MISLEADING vs the merged code
(a stale table name, a command that does not exist, a wrong default, a wrong ADR status, a "does
X" the code does not do). No style nitpicks unless misleading.
Severity: high (a wrong claim a user/operator would act on and be broken by, e.g. wrong
command/flag/deploy step), medium (misleading/ambiguous/stale), low (typo/broken link/minor).

DIMENSION: verify the ADR "implementation status" notes added to docs/adr/0007, 0009, 0010,
0011, 0012. Check each status note reflects reality: 0009 selective scanning (opt-in default,
/watch commands, the #41 watched-clobber fix) ? does the note claim it is implemented as
designed, and is that true? 0011 provider-neutral schema (tables actually created, deprecated

tables, dedup) ? accurate? 0012 F1-F7 pipeline status (which stages exist, deep-inspection fetcher NOT wired, no config command for policy keys, no retention cleanup) ? does the note overstate completeness? 0010 identity/session split; 0007 inline commands. Flag any status note that overstates or misstates what was built.

2. assistant (2026-07-07T03:35:18.948Z)

I'll start by reading the doc diff and cross-checking claims against the merged code. Let me begin by identifying the files in the PR and reading the diff.

3. user (2026-07-07T03:35:21.069Z)

```
Dockerfile | 3 +-
README.md | 195 ++++++---
docs/adr/0007-inline-config-commands.md | 9 +
docs/adr/0009-selective-channel-scanning.md | 10 +
docs/adr/0010-identity-and-session-separation.md | 9 +-
docs/adr/0011-provider-neutral-domain-schema.md | 22 +-
docs/adr/0012-policy-pipeline.md | 70 ++--
docs/design.md | 473 ++++++-----
docs/plan.md | 334 +++++-----
docs/runbook.md | 416 ++++++-----
pyproject.toml | 2 +-
src/tg_video_filter/command_service.py | 4 +-
src/tg_video_filter/models.py | 20 +-
tests/test_commands.py | 2 +-
14 files changed, 782 insertions(+), 787 deletions(-)
```

4. assistant (2026-07-07T03:35:23.447Z)

Let me read the doc diffs. I'll start with the ADR files and the source edits, which are the highest-signal for accuracy.

5. user (2026-07-07T03:35:25.999Z)

```
diff --git a/docs/adr/0007-inline-config-commands.md b/docs/adr/0007-inline-config-commands.md
index 4a40af4..f9bbfbb 100644
--- a/docs/adr/0007-inline-config-commands.md
+++ b/docs/adr/0007-inline-config-commands.md
@@ -4,6 +4,15 @@
- **Date**: 2026-07-04
- **Decision owner**: project owner

+## Implementation status
+
+The original command set has been extended. Current Saved Messages commands
+include `/show config`, `/show commands`, the `/set` size/length/min-resolution
+and exclusion commands, source scanning commands (`/channels`, `/watch`,
+`/unwatch`, `/watchlist`), backfill commands, and `/why <content_item_id>`.
+`CommandService` is provider-independent and the Telethon module is only a thin
+adapter.
+
+## Context

Filter criteria (duration bounds, resolution threshold, max file size) were set
diff --git a/docs/adr/0009-selective-channel-scanning.md b/docs/adr/0009-selective-channel-scanning.md
index d03db20..5fe405c 100644
--- a/docs/adr/0009-selective-channel-scanning.md
+++ b/docs/adr/0009-selective-channel-scanning.md
@@ -4,6 +4,16 @@
- **Date**: 2026-07-04
- **Decision owner**: project owner
```

```

+## Implementation status
+
+Selective live scanning is implemented on the provider-neutral `sources`
+table, not the legacy `channels` table. Unknown live sources are created with
+`watched=false` and skipped until the user runs `/watch`. The shipped
+`/channels` command lists discovered source rows; `/backfill` can also discover
+historical dialogs. The provider-neutral `routing_rules` table exists, but the
+runtime still delivers through the account's current `delivery_target` unless
+future routing logic is added.
+
+## Context

Today the userbot scans every channel/group the account is a member of.
diff --git a/docs/adr/0010-identity-and-session-separation.md b/docs/adr/0010-identity-and-
session-separation.md
index 7791bc2..1c826cd 100644
--- a/docs/adr/0010-identity-and-session-separation.md
+++ b/docs/adr/0010-identity-and-session-separation.md
@@ -1,11 +1,18 @@
# ADR 0010: Separate product identity from Telegram sessions

-- Status: Proposed (awaiting decision owner pick between Option A and Option B)
+ Status: Accepted and implemented (Option A; Track A merged)
- Date: 2026-07-05
- Decision owner: project owner
- Tracking issue: #26 (Track A of the provider-neutral umbrella, #25)
- Blocks: ADR 0008 (Bot API onboarding), #27 (Track B domain schema), #28 (Track C
delivery), #29 (Track D commands)

+## Implementation status
+
+The schema now has `accounts` for product identity and `telegram_sessions` for
+runnable MTPROTO sessions. Legacy `users` APIs remain compatibility shims over
+those tables, so older call sites and databases continue to work while new
+provider-neutral tables use `account_id`.
+
+## Context

Today the `users` table conflates two orthogonal concepts:
diff --git a/docs/adr/0011-provider-neutral-domain-schema.md b/docs/adr/0011-provider-neutral-
domain-schema.md
index aa37418..3be70a1 100644
--- a/docs/adr/0011-provider-neutral-domain-schema.md
+++ b/docs/adr/0011-provider-neutral-domain-schema.md
@@ -7,6 +7,16 @@
- Depends on: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
- Blocks: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly #30
(Track E)

+## Implementation status
+
+The provider-neutral schema is implemented. `sources`, `content_items`,
+`policies`, `destinations`, `routing_rules`, `delivery_attempts`, and
+`policy_decisions` exist in the current SQLite schema. Legacy `channels`,
+`videos`, and `filter_config` APIs are compatibility shims over the newer
+tables. The Telegram path now captures caption text into `content_items.text`;
+`content_items.link_url` remains available for future link/article content,
+while URL policy stages currently extract links from captured text.
+
+## Context

ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
@@ -374,9 +384,9 @@ shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
single-purpose, but the surface area is larger.
- The dedup-key derivation logic is in code, not the schema ? needs its

```

```
own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
-- Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
- `text`) that aren't populated until later tracks. This is intentional
- (avoid future migrations) but means some columns are NULL until then.
+- `content_items.link_url` is still reserved for future link/article content.
+ The current Telegram video path captures captions in `content_items.text`
+ and URL policy stages extract links from that captured text.
```

Alternatives considered

```
@@ -399,9 +409,9 @@ shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
rejected), or 1C (provider-assigned only).
2. **Question 2** ? `Policy` ? `filter_config`: 2A (generalize, recommended)
or 2B (wrap, rejected).
-3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
- / `text` as NULL until later tracks populate them? Recommendation: yes
- (avoids a future `ALTER TABLE`).
+3. **Dead columns in Phase 1** ? resolved by later tracks for
+ `content_items.text`; `link_url` remains reserved for future link/article
+ content.
4. **Deprecated-table retention** ? one release (default), matching ADR 0010?
Recommendation: yes.
```

```
diff --git a/docs/adr/0012-policy-pipeline.md b/docs/adr/0012-policy-pipeline.md
index d5d86d1..ae0b8b3 100644
```

```
--- a/docs/adr/0012-policy-pipeline.md
+++ b/docs/adr/0012-policy-pipeline.md
```

```
@@ -1,11 +1,27 @@
```

```
# ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates
```

```
-- **Status**: Proposed
+- **Status**: Accepted and implemented (F1-F7 merged)
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)
```

Implementation status

```
+
+This ADR is implemented through the F1-F7 PR sequence tracked by umbrella
+issue #42:
+
```

```
+- #50 ? policy engine prototype and config persistence.
+- #51 ? topic preferences and caption capture.
+- #52 ? caption safety.
+- #53 ? URL/domain reputation.
+- #54 ? deep-inspection budget boundary.
+- #55 ? `policy_decisions` audit table.
+- #57 ? `/why <content_item_id>` explainability command.
+
```

```
+Follow-up retention cleanup for `policy_decisions` is tracked separately in
+issue #56.
+
```

Context

```
The system today evaluates every matched video against a single `FilterConfig`
@@ -25,9 +41,9 @@ them, and defer expensive work until after cheap rejections.
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
-| Config persistence shims | `load_filter_config` / `save_filter_config` parse and write only
the flat `FilterConfig` shape. Unknown future keys are not preserved by this path. |
-| `content_items.text` / `link_url` | Columns exist from ADR 0011, but the live Telegram path
```

does not populate captions or extracted links yet. |

- | Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan. |
- +| Config persistence shims | `load\_filter\_config` / `save\_filter\_config` merge flat `FilterConfig` updates into the default policy JSON so unknown policy keys survive legacy `/set` commands. |
- +| `content\_items.text` / `link\_url` | The Telegram path captures caption text in `content\_items.text`. `link\_url` remains available for future link/article content; URL reputation currently extracts URLs from captured text at evaluation time. |
- +| Security / topic / deep-inspect | Topic preferences, caption safety, URL/domain reputation, and the deep-inspection budget boundary are implemented. No production deep-inspection fetcher is wired by default. |

What we want

@@ -214,30 +230,23 @@ keys inside that blob:

- `policies.config\_json` gains optional `topics`, `security\_checks`, and `deep\_inspect` keys.
- The legacy `load\_filter\_config` / `save\_filter\_config` shim is not a safe persistence path for those keys. It parses through `FilterConfig` and then serializes `FilterConfig` again, so unknown keys would be dropped. F1 must add a policy-config persistence path that reads/writes the raw JSON object (or a typed superset model) and preserves unknown keys when legacy `/set` commands update flat `FilterConfig` fields.
- Future `policy\_decisions` table (F6) stores the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.
-
- ### Implementation phasing (future issues)
-
- | Phase | Scope | Dependencies |
- |---|---|---|
- | **F1** ? Policy engine prototype | `PolicyStage` protocol, `evaluate\_pipeline()`, raw/typed policy-config persistence, and hot-path wiring from live handler to `ContentItem`/`Source`/`Policy` context | Tracks B-E landed |
- | **F2** ? Topic preferences | `topics` config parsing, source title matching, caption capture into `content\_items.text`, caption keyword matching | F1 |
- | **F3** ? Caption safety | Unsafe caption term/pattern scanning from captured text | F1, F2 caption capture |
- | **F4** ? URL/domain reputation | Caption link extraction into `content\_items.link\_url` or stage metadata, domain blacklist checks with no page fetch | F1, F2 caption capture |
- | **F5** ? Deep inspection | `ContentFetcher`, ffprobe/perceptual hash, download budget enforcement | F1, accepted ADR 0003 amendment above |
- | **F6** ? Policy decisions table | Persistent audit trail storage for accepted and rejected items | F1 |
- | **F7** ? Explainability endpoint | `/why <content\_item\_id>` command showing the explanation trace | F6 |
-
- Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **design contract** the phases code against.
- + The legacy `load\_filter\_config` / `save\_filter\_config` shim now preserves + unknown policy keys when legacy `/set` commands update flat `FilterConfig` + fields.
- + `policy\_decisions` stores one row per `(content\_item\_id, policy\_id)` with + final pass/fail, ordered stage trace JSON, and `evaluated\_at`.
- +
- ##### Implementation phasing
- +
- +| Phase | Scope | PR | Status |
- +|---|---|---|---|
- +| **F1** ? Policy engine prototype | `PolicyStage`, `evaluate\_pipeline()`, config preservation, live hot-path wiring | #50 | Merged |
- +| **F2** ? Topic preferences | `topics` parsing, source-title/caption matching, caption capture | #51 | Merged |

```
+| **F3** ? Caption safety | Unsafe caption term/pattern scanning | #52 | Merged |
+| **F4** ? URL/domain reputation | Captured-text URL extraction and domain blocklist checks
with no page fetch | #53 | Merged |
+| **F5** ? Deep inspection | `ContentFetcher` protocol and budget boundary | #54 | Merged |
+| **F6** ? Policy decisions table | Persistent audit trail storage for accepted and rejected
policy-stage decisions | #55 | Merged |
+| **F7** ? Explainability endpoint | `/why <content_item_id>` command showing stored traces and
safe fallback output | #57 | Merged |
```

Consequences

@@ -265,8 +274,9 @@ only defines the **design contract** the phases code against.

- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Caption and link stages require ingestion changes because the live path does not currently write `content\_items.text` or `content\_items.link\_url`.
- + Caption capture is now wired through `content\_items.text`; URL reputation extracts links from captured text. `content\_items.link\_url` remains reserved for future link/article content.
- Deep inspection amends ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

6. assistant (2026-07-07T03:35:29.211Z)

Now let me read the source code to verify these claims. Let me look at the key files in parallel.

7. user (2026-07-07T03:35:30.820Z)

```
__init__.py
__main__.py
__pycache__
backfill.py
bootstrap.py
command_service.py
commands.py
config.py
db.py
delivery.py
filters
manager.py
models.py
policy.py
telethon_client.py
```

8. user (2026-07-07T03:35:32.600Z)

```
diff --git a/Dockerfile b/Dockerfile
index 9f4ffa3..a0c882b 100644
--- a/Dockerfile
+++ b/Dockerfile
@@ -25,6 +25,5 @@ ENV DB_PATH=/data/app.db \
    SESSIONS_DIR=/data/sessions \
    LOG_LEVEL=INFO
```

```
-# Run the manager entrypoint (Phase 4). For now this will fail fast until
-# __main__.py lands; that's expected at the Phase 0 stage.
+# Run the manager entrypoint.
CMD ["python", "-m", "tg_video_filter"]
diff --git a/pyproject.toml b/pyproject.toml
index 14ea202..83c4aad 100644
--- a/pyproject.toml
+++ b/pyproject.toml
```

```

@@ -5,7 +5,7 @@ build-backend = "hatchling.build"
[project]
name = "tg-video-filter"
version = "0.1.0"
-description = "Telethon userbot that filters Telegram channel videos by duration, resolution,
and size, and forwards matches to Saved Messages."
+description = "Telethon userbot that filters selected Telegram video sources, delivers matches,
and explains policy decisions."
readme = "README.md"
requires-python = ">=3.11"
license = { text = "TBD" }
diff --git a/src/tg_video_filter/command_service.py b/src/tg_video_filter/command_service.py
index 0ab172a..79fef65 100644
--- a/src/tg_video_filter/command_service.py
+++ b/src/tg_video_filter/command_service.py
@@ -150,11 +150,11 @@ def format_commands() -> str:
    "/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
        "    or `/set min res <N>p` ? same as above",
        """
-        "? **Content exclusions** (all off by default)",
+        "? **Content exclusions**",
        "/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
        "/set exclude round on|off` ? Exclude circular video messages",
        "/set exclude nosound on|off` ? Exclude videos without audio\n"
-        "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
+        "    Defaults: GIF/round excluded, nosound allowed.",
        """
        "? **Backfill**",
        "/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
90d)",
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index fd4c85e..5c36b88 100644
--- a/src/tg_video_filter/models.py
+++ b/src/tg_video_filter/models.py
@@ -149,9 +149,10 @@ class Source(BaseModel):
    class ContentItem(BaseModel):
        """A normalized content record discovered from any :class:`Source` (ADR 0011).

-        Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
-        (`content_type='video'`); `text` / `link_url` are landed now but stay
-        NULL until later tracks populate them (avoids a future `ALTER TABLE`).
+        Generalizes the legacy :class:`VideoRecord`. The Telegram path stores videos
+        today (`content_type='video'`) and captures caption text; `link_url` is
+        available for future link/article content while URL policy stages currently
+        extract links from captured text at evaluation time.
        `dedup_key` is the provider-neutral dedup key (Option 1A: tiered lookup
        wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
        """

@@ -198,9 +199,7 @@ class Policy(BaseModel):
    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    name: str = Field(description="Policy name; 'default' for the account default policy")
    config_json: str = Field(
-        description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
-    )
+        description="FilterConfig JSON blob plus ADR 0012 policy keys"
    is_default: bool = Field(
        default=False, description="Exactly one per account (UNIQUE partial index)"
    )

@@ -313,11 +312,12 @@ class PolicyDecision(BaseModel):

class FilterConfig(BaseModel):
-    """Per-user filter configuration.

```

```
+ """Flat per-account filter configuration.

- v1 rules: duration bounds, minimum height (resolution), and max file size.
- Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
- new field later is backwards-compatible.
+ Stored inside the account's default `policies.config_json` object through
+ compatibility shims. Legacy `filter_config` callers still work, but new
+ policy keys can live alongside these flat fields without being erased by
+ `/set` command updates.
+ """
```

```
    # Duration bounds in seconds. None means unbounded on that side.
diff --git a/tests/test_commands.py b/tests/test_commands.py
index 162c97b..7ff316d 100644
--- a/tests/test_commands.py
+++ b/tests/test_commands.py
@@ -1101,7 +1101,7 @@ def test_format_commands_includes_exclude() -> None:
    assert "/set exclude gif" in result
    assert "/set exclude round" in result
    assert "/set exclude nosound" in result
-    assert "off by default" in result
+    assert "GIF/round excluded, nosound allowed" in result
```

```
# --- regex: /set min resolution -----
```

9. assistant (2026-07-07T03:35:35.002Z)

Now let me verify the key ground-truth claims against the code. Let me check FilterConfig defaults, the policy pipeline, should_ingest_source, and the tables.

10. user (2026-07-07T03:35:37.967Z)

```
exclude_gif True
exclude_round True
exclude_nosound False
```

11. user (2026-07-07T03:35:38.509Z)

```
46:     """Raised when a deep-inspection budget is exhausted."""
144:     stage_name: str
148:     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
152:         stage_name=stage_name,
157:     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
161:         stage_name=stage_name,
374:     stage_name = "topic-preferences"
386:         self.stage_name,
391:         return StageResult.pass_(self.stage_name, "topic preferences disabled")
397:         self.stage_name,
403:         return StageResult.pass_(self.stage_name, "no include topics configured")
409:         self.stage_name,
413:         return StageResult.fail(self.stage_name, "no include topics matched")
418:         self.stage_name,
424:         self.stage_name,
505:     stage_name = "url-reputation"
517:         self.stage_name,
532:         self.stage_name,
536:         return StageResult.pass_(self.stage_name, "domain reputation disabled")
543:         self.stage_name,
552:         self.stage_name,
561:         self.stage_name,
585:     stage_name = "caption-safety"
597:         self.stage_name,
610:         self.stage_name,
```

```

614:         return StageResult.pass_(self.stage_name, "caption safety disabled")
629:         self.stage_name,
640:         self.stage_name,
671:         self.stage_name,
684:         self.stage_name,
724:     stage_name = "deep-inspection"
736:         self.stage_name,
741:         return StageResult.pass_(self.stage_name, "deep inspection disabled")
749:         self.stage_name,
759:         self.stage_name,
772:         self.stage_name,
783:         DeepInspectionStage.stage_name,
789:         DeepInspectionStage.stage_name,
799:         DeepInspectionStage.stage_name,
806:         DeepInspectionStage.stage_name,
813:         DeepInspectionStage.stage_name,
823:async def evaluate_pipeline(
841:     "stage_name": result.stage_name,

```

12. assistant (2026-07-07T03:35:41.031Z)

FilterConfig defaults confirmed. Let me verify the pipeline order and the ADR claims. Let me look at evaluate_pipeline more closely, and check should_ingest_source and tables.

13. user (2026-07-07T03:35:42.514Z)

```

823     async def evaluate_pipeline(
824         ctx: PolicyContext,
825         stages: list[PolicyStage],
826     ) -> tuple[bool, list[StageResult]]:
827         """Evaluate stages in order and short-circuit on the first failure."""
828         results: list[StageResult] = []
829         for stage in stages:
830             result = await stage.evaluate(ctx)
831             results.append(result)
832             if not result.passed:
833                 return False, results
834         return True, results
835
836
837     def stage_results_to_trace(results: list[StageResult]) -> list[dict[str, object]]:
838         """Convert in-memory stage results to a JSON-serializable trace."""
839         return [
840             {
841                 "stage_name": result.stage_name,
842                 "passed": result.passed,
843                 "reason": result.reason,
844                 "metadata": result.metadata,
845             }
846             for result in results
847         ]
848
849
850     __all__ = [
851         "CaptionSafetyConfig",
852         "CaptionSafetyStage",
853         "ContentFetcher",
854         "DeepInspectionStage",
855         "DomainReputationConfig",
856         "ExtractedUrl",
857         "PolicyContext",
858         "PolicyBudget",
859         "PolicyBudgetExceeded",
860         "PolicyStage",
861         "StageResult",

```

```

862     "TopicPreferenceStage",
863     "TopicPreferences",
864     "UrlReputationStage",
865     "evaluate_pipeline",
866     "extract_caption_urls",
867     "load_caption_safety_config",
868     "load_deep_inspection_enabled",
869     "load_domain_reputation_config",
870     "load_topic_preferences",
871     "normalize_domain",
872     "parse_caption_safety_config",
873     "parse_domain_reputation_config",
874     "parse_topic_preferences",
875     "stage_results_to_trace",
876 ]
877

```

14. user (2026-07-07T03:35:43.782Z)

```

104:     watched          INTEGER NOT NULL DEFAULT 0,
537:     # 1a. Backfill sources from channels first (known title + watched=1). Order
538:     #     matters: channels-first so title/watched data wins; the videos-based
543:     #     (account_id, provider, provider_source_id, title, kind, watched,
554:     #     ? the channels-based row above survives with its title + watched=1.
558:     #     (account_id, provider, provider_source_id, kind, watched, seen_first_at)
884:     watched=bool(row["watched"]),
1151:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1157:     ``watched`` is **user-controlled state** (toggled only via
1158:     :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is
1161:     so clobbering ``watched`` here would silently un-watch a channel the user
1162:     opted into after its first ingested video (ADR 0009). ``watched`` is only set
1169:     (account_id, provider, provider_source_id, title, kind, watched,
1181:     int(source.watched),
1597:     watched=False,
1733:async def set_source_watched(
1734:     conn: aiosqlite.Connection, account_id: int, source_id: int, watched: bool
1736:     """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
1738:     "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
1739:     (int(watched), source_id, account_id),
1745:     conn: aiosqlite.Connection, account_id: int, *, watched_only: bool = False
1747:     """List all sources for an account, optionally filtering to watched only."""
1751:     if watched_only:
1752:         query += " AND watched = 1"
1753:     query += " ORDER BY watched DESC, provider_source_id ASC"
1759:async def should_ingest_source(
1764:     - Absent sources row: create it lazily with ``watched=False`` for
1765:     ``/channels`` discoverability; return False (not yet watched).
1766:     - Existing row: respect the ``watched`` flag.
1773:     # unwatched row so /channels can list it (ADR 0009 lines 48-54).
1781:     watched=False,
1785:     return src.watched
1803:     watched=False,
1985:     "set_source_watched",

```

15. assistant (2026-07-07T03:35:46.619Z)

The core ground truth is confirmed. Now let me verify the pipeline STAGES order is actually built in that order in the live path, and check the deep-inspection fetcher wiring. Let me find where the pipeline stages are assembled and where evaluate_pipeline is called in the live handler.

16. user (2026-07-07T03:35:48.353Z)

```

48:     CaptionSafetyStage,

```

```

49:     DeepInspectionStage,
52:     TopicPreferenceStage,
53:     UrlReputationStage,
54:     evaluate_pipeline,
452:     # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
453:     # user-facing enable command must land with fetcher/budget wiring.
454:     passed, results = await evaluate_pipeline(
457:         TopicPreferenceStage(),
458:         UrlReputationStage(),
459:         CaptionSafetyStage(),
460:         DeepInspectionStage(),

```

17. assistant (2026-07-07T03:35:51.075Z)

Pipeline order confirmed: topic-preferences -> url-reputation -> caption-safety -> deep-inspection. Let me check the context around line 452 to confirm fetcher/budget are None in the live path.

18. user (2026-07-07T03:35:51.822Z)

```

430         document_id=str(document_id) if document_id is not None else None,
431         source_id=source.id,
432         provider_message_id=str(message_id),
433         size_bytes=meta.size_bytes,
434         duration_s=meta.duration_s,
435         height=meta.height,
436         width=meta.width,
437     ),
438     matched=matched,
439 )
440
441
442 async def _evaluate_policy_stages(
443     conn: aiosqlite.Connection,
444     account_id: int,
445     source: Source,
446     content_item: ContentItem,
447 ) -> tuple[bool, list[StageResult], int | None]:
448     """Evaluate ADR 0012 policy stages for a candidate item."""
449     policy = await db.get_default_policy(conn, account_id)
450     if policy is None:
451         return True, [], None
452     # DeepInspectionStage is dormant unless ``deep_inspect`` is enabled. Any
453     # user-facing enable command must land with fetcher/budget wiring.
454     passed, results = await evaluate_pipeline(
455         PolicyContext(content_item=content_item, source=source, policy=policy),
456         [
457             TopicPreferenceStage(),
458             UrlReputationStage(),
459             CaptionSafetyStage(),
460             DeepInspectionStage(),
461         ],
462     )
463     return passed, results, policy.id
464
465
466 async def _record_policy_decision(
467     conn: aiosqlite.Connection,
468     *,
469     inserted: VideoRecord,
470     policy_id: int | None,
471     policy_passed: bool,
472     policy_results: list[StageResult],
473 ) -> None:
474     """Persist a policy trace after the content item has been inserted."""

```

19. assistant (2026-07-07T03:35:55.219Z)

Confirmed: `PolicyContext` is created without fetcher/budget (they default to None), so deep inspection has no production fetcher. The ADR 0012 status note says "No production deep-inspection fetcher is wired by default" ? accurate.

Now let me read the larger doc files (README, design, plan, runbook) diffs to check concrete claims. These are big; let me focus on concrete factual claims.

20. user (2026-07-07T03:35:56.996Z)

```
diff --git a/README.md b/README.md
index 80de756..d4c6ecc 100644
--- a/README.md
+++ b/README.md
@@ -1,75 +1,148 @@
 # telegram-video-filter
```

```
-A Telethon-based **userbot** that logs in as your own Telegram account, watches
-new video messages across all the channels/groups you subscribe to, applies
-configurable filters (duration, resolution, file size), and **forwards matching
-videos to your Saved Messages** in real time.
-
-> ?? This project automates a **user** account (a "userbot") via MTProto. Keep
-> volumes reasonable and respect Telegram's Terms of Service. You are
-> responsible for your own account. See [docs/adr/0001-userbot-over-
-botapi.md](docs/adr/0001-userbot-over-botapi.md).
-
-## Status
-
-? **v1 complete** ? all 6 build phases merged via PR. The app runs end-to-end:
-bootstrap a user account ? manager watches their channels ? filters videos ?
-forwards matches to Saved Messages. See [docs/plan.md](docs/plan.md) for the
-build history and [docs/runbook.md](docs/runbook.md) to deploy.
-phased build plan and current status.
-
-## What it does (v1)
-
-- Watches every channel/group your account is a member of ? no per-channel setup.
-- Reads video metadata **without downloading** the file (duration, width/height,
- size, MIME).
-- Applies an **AND** of your filters, e.g.:
- - duration between 5s and 2h,
- - height ? 1080 (1080p or better),
- - size ? 2 GiB.
-- Forwards each match to your **Saved Messages** in real time, or to a
- **channel you own** if you set `delivery_target` (per-user).
-- **Never forwards the same video twice** ? three-tier content dedup
- (message instance ? Telegram `document.id` ? metadata fingerprint) means
- a video cross-posted to 5 channels is delivered once. See
- [docs/adr/0006-content-dedup.md](docs/adr/0006-content-dedup.md).
-- Supports multiple users (<100) on a single host, each with its own session,
- filters, delivery mode/target, and dedup state.
-
-## What it does NOT do yet (v2+)
-
-- Public-channel `/add @channel` join, manual-add ingestion.
-- Scheduled digest delivery and on-demand `/list` / `/search`.
-- ffmpeg-based deep inspection (`/inspect`).
-- Codec / keyword / MIME filters.
+A Telethon-based Telegram **userbot** that logs in as your own account, filters
+video messages from selected sources, and delivers matches to Saved Messages or
+another Telegram destination you control.
+
+The app is metadata-first: it reads Telegram message metadata and captions
```

```

+without downloading media for the normal path. It is built for a small
+multi-user deployment on one host, with SQLite state and file-backed Telethon
+sessions under `data/`.
+
+> This project automates a **user** account via MTProto. Keep volumes
+> reasonable and respect Telegram's Terms of Service. Session files are
+> credentials. See [ADR 0001](docs/adr/0001-userbot-over-botapi.md).
+
+### Current Status
+
+The main userbot path is implemented end to end:
+
+- Bootstrap one or more Telegram user sessions.
+- Discover Telegram channels/groups as sources.
+- Opt in live ingestion per source with `/watch`.
+- Filter video metadata with duration, resolution, size, GIF/round/nosound
+  exclusions, and the ADR 0012 policy pipeline.
+- Deliver accepted videos by forward, link, or both.
+- Persist content, delivery state, policy decisions, and explanation traces.
+- Explain historical decisions with `/why <content_item_id>`.
+
+The original v1 plan and the ADR 0012 F1-F7 policy pipeline are complete. The
+current follow-up is policy-decision retention cleanup; see issue #56.
+
+### Core Features
+
+- **Selective live scanning**: unknown sources are discovered as `watched=false`.
+  Use `/channels`, `/watch <id|@name>`, `/unwatch <id|@name>`, and `/watchlist`
+  from Saved Messages to control live ingestion.
+- **Metadata filters**: duration bounds, minimum height, maximum size, GIF,
+  round-video, and nosound exclusions.
+- **Policy stages**: topic preferences, captured-caption safety checks,
+  captured URL/domain reputation checks, and an opt-in deep-inspection boundary.
+  Deep inspection is budgeted but no production fetcher is wired by default.
+- **Deduplication**: provider-neutral dedup keys catch same message, same
+  Telegram `document.id`, and same metadata fingerprint.
+- **Delivery modes**: `forward`, `link`, or `both`, targeting `"me"`,
+  `@username`, or a numeric Telegram chat id.
+- **Backfill**: `/backfill [N][h|d|w]` scans historical channel/group messages.
+  Backfill is intentionally broad and should be run carefully on large accounts.
+- **Explainability**: `/why <content_item_id>` renders the stored policy trace
+  when available, or a clearly caveated current metadata replay for older
+  items without a policy-decision row.
+
+### What Is Not Built
+
+- Bot API onboarding/front-door commands from ADR 0008.
+- Public-channel auto-join or manual `/add @channel` ingestion.
+- Scheduled digests and on-demand search/list commands.
+- A production deep-inspection fetcher/downloader.
+- Policy-decision retention cleanup.
+- Horizontal scaling across multiple hosts.
+
+### Quick Start
+
+## Documentation
+```bash
+cp .env.example .env
+# edit .env with TG_API_ID and TG_API_HASH from https://my.telegram.org
+
+docker compose up -d --build
+python -m tg_video_filter.bootstrap
+docker compose restart app
+```
+

```

```

+After startup, send commands to your own Saved Messages:
+
+```text
+/channels
+/watch <source-id-or-title>
+/show config
+/show commands
+```
+
+See [docs/runbook.md](docs/runbook.md) for the full operational workflow.

-- [docs/plan.md](docs/plan.md) ? phased build plan, scope, verification.
-- [docs/design.md](docs/design.md) ? architecture, data model, control flow.
-- [docs/runbook.md](docs/runbook.md) ? deploy, onboard users, rotate
- sessions, handle flood waits.
-- [docs/adr/](docs/adr/) ? Architecture Decision Records:
- - [0001 ? Userbot over Bot API](docs/adr/0001-userbot-over-botapi.md)
- - [0002 ? SQLite over PostgreSQL](docs/adr/0002-sqlite-over-postgres.md)
- - [0003 ? Metadata-first probing](docs/adr/0003-metadata-first-probing.md)
- - [0004 ? Pluggable delivery modes](docs/adr/0004-pluggable-delivery-modes.md)
- - [0005 ? Per-user delivery target](docs/adr/0005-per-user-delivery-target.md)
+## Useful Commands

-## Quick start
+Filter and config:

-See the [runbook](docs/runbook.md) for the full guide. In short:
+- `/show config`
+- `/show commands`
+- `/set size <N><unit>` where unit is `KB`, `MB`, `GB`, or `TB`
+- `/set length <N><unit>` or `/set max length <N><unit>`
+- `/set min length <N><unit>`
+- `/set min resolution <N>p` or `/set min res <N>p`
+- `/set exclude gif|round|nosound on|off`
+
+Source scanning:
+
+- `/channels`
+- `/watch <id|@name|title-fragment>`
+- `/unwatch <id|@name|title-fragment>`
+- `/watchlist`
+
+Backfill and explainability:
+
+- `/backfill`, `/backfill 8h`, `/backfill 7d`, `/backfill 2w`
+- `/stop`
+- `/why <content_item_id>`
+
+## Documentation
+
+- [docs/runbook.md](docs/runbook.md) - deployment, onboarding, commands,
+ operations, and troubleshooting.
+- [docs/design.md](docs/design.md) - current runtime architecture, data model,
+ control flow, and policy pipeline.
+- [docs/plan.md](docs/plan.md) - implementation status and completed tracks.
+- [docs/adr/](docs/adr/) - decision history:
+ - [0001 - Userbot over Bot API](docs/adr/0001-userbot-over-botapi.md)
+ - [0002 - SQLite over PostgreSQL](docs/adr/0002-sqlite-over-postgres.md)
+ - [0003 - Metadata-first probing](docs/adr/0003-metadata-first-probing.md)
+ - [0004 - Pluggable delivery modes](docs/adr/0004-pluggable-delivery-modes.md)
+ - [0005 - Per-user delivery target](docs/adr/0005-per-user-delivery-target.md)
+ - [0006 - Content dedup](docs/adr/0006-content-dedup.md)
+ - [0007 - Inline config commands](docs/adr/0007-inline-config-commands.md)
+ - [0008 - Bot API onboarding front door](docs/adr/0008-bot-api-onboarding-front-door.md)
+ - [0009 - Selective channel scanning](docs/adr/0009-selective-channel-scanning.md)

```

```

+ - [0010 - Identity and session separation](docs/adr/0010-identity-and-session-separation.md)
+ - [0011 - Provider-neutral domain schema](docs/adr/0011-provider-neutral-domain-schema.md)
+ - [0012 - Policy pipeline](docs/adr/0012-policy-pipeline.md)
+
+## Development

    ``bash
-cp .env.example .env          # set TG_API_ID, TG_API_HASH from my.telegram.org
-docker compose up -d --build
-python -m tg_video_filter.bootstrap # interactive one-time login per user
-docker compose restart app
+python -m venv .venv
+.\.venv\Scripts\python.exe -m pip install -e .[dev]
+.\.venv\Scripts\python.exe -m ruff check src tests
+.\.venv\Scripts\python.exe -m pytest
    ``

-## Tech stack
+The test suite uses an 80% coverage gate. Live Telethon glue and interactive
+entrypoints are marked `pragma: no cover` and covered by focused pure-unit
+tests around their helpers.
+
+## Tech Stack

-Python 3.11+ · [Telethon](https://docs.telethon.dev/) · aiosqlite (SQLite) ·
-pydantic / pydantic-settings · Docker · pytest · ruff.
+Python 3.11+ · Telethon · aiosqlite/SQLite · pydantic/pydantic-settings ·
+Docker · pytest · ruff.

## License

```

21. assistant (2026-07-07T03:36:00.277Z)

Now let me verify several concrete README claims: the `/watch`/`/unwatch` argument forms (id|@name|title-fragment), the `/stop` command, `/set size` units, bootstrap invocation, and the delivery modes. Let me check the commands.

22. user (2026-07-07T03:36:02.504Z)

No matches found

23. user (2026-07-07T03:36:06.092Z)

```

38:_WHY_RE = re.compile(r"^/why(?:\s+(.))?$", re.IGNORECASE)
46:_WATCH_RE = re.compile(r"^/watch\s+(.)?$ ", re.IGNORECASE)
47:_UNWATCH_RE = re.compile(r"^/unwatch\s+(.)?$ ", re.IGNORECASE)
48:_CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
49:_WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
159:         "? **Backfill**",
160:         "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
90d)",
161:         "`/stop` ? Cancel running backfill",
164:         "`/channels` ? List all known sources with watch status",
165:         "`/watch <id|@name>` ? Start watching a source for new videos",
166:         "`/unwatch <id|@name>` ? Stop watching a source",
167:         "`/watchlist` ? Show only watched sources",
170:         "`/why <content_item_id>` ? Explain why an item passed or failed",
436:         m = _WATCH_RE.match(text)
438:         return await self._handle_watch(conn, user_id, m.group(1).strip(), current_cfg)
440:         m = _UNWATCH_RE.match(text)
442:         return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
444:         if _CHANNELS_RE.match(text):
445:             return await self._handle_channels(conn, user_id)

```

```

447:         if _WATCHLIST_RE.match(text):
448:             return await self._handle_watchlist(conn, user_id)
450:         # --- /why <content_item_id> -----
464:     async def _handle_watch(
467:         """Parse ``/watch <@username|channel_id>`` and toggle ``watched=1``."""
471:         f"? Source `{target}` not found. Use `/channels` to list "
476:         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-type]
478:         return CommandResult(f"? Now watching **{label}** (`{src.provider_source_id}`)")
480:     async def _handle_unwatch(
483:         """Parse ``/unwatch <@username|channel_id>`` and toggle ``watched=0``."""
487:         f"? Source `{target}` not found. Use `/channels` to list sources."
489:         await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-type]
491:         return CommandResult(f"? Stopped watching **{label}**
(`{src.provider_source_id}`)")
493:     async def _handle_channels(self, conn: aioredis.Connection, user_id: int) ->
CommandResult:
494:         """List all known sources with watch status."""
500:         "channel. Run `/backfill` to discover historical channels, "
505:         icon = "?" if s.watched else "?"
510:     async def _handle_watchlist(self, conn: aioredis.Connection, user_id: int) ->
CommandResult:
511:         """List only watched sources."""
512:         sources = await db.list_account_sources(conn, user_id, watched_only=True)
515:         "? **No watched sources.**\n\n"
516:         "Use `/channels` to see available sources, then `/watch <id>` "
519:         lines = ["? **Watched sources**", ""]
534:         return CommandResult("? Usage: `/why <content_item_id>`)
537:         return CommandResult("? Usage: `/why <content_item_id>`)

```

24. assistant (2026-07-07T03:36:08.989Z)

Commands confirmed. The README mentions `/watch <id|@name|title-fragment>`. Let me verify whether title-fragment resolution is actually supported by checking `\_resolve\_source` (the lookup used by `\_handle\_watch`).

25. user (2026-07-07T03:36:11.026Z)

```

475-         )
476-         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-type]
477-         label = src.title or src.provider_source_id
478-         return CommandResult(f"? Now watching **{label}** (`{src.provider_source_id}`)")
479-
480-     async def _handle_unwatch(
481-         --
482-         )
483-         await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-type]
484-         label = src.title or src.provider_source_id
485-         return CommandResult(f"? Stopped watching **{label}**
(`{src.provider_source_id}`)")
486-
487-     async def _handle_channels(self, conn: aioredis.Connection, user_id: int) ->
CommandResult:
488-         --
504-         for s in sources:
505-             icon = "?" if s.watched else "?"
506-             label = s.title or s.provider_source_id
507-             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
508-         return CommandResult("\n".join(lines))
509-
510-         --
511-         lines = ["? **Watched sources**", ""]
512-         for s in sources:
513-             label = s.title or s.provider_source_id
514-             lines.append(f"? `{s.provider_source_id}` ? {label}")
515-         return CommandResult("\n".join(lines))

```

```

524-
--
547-         return CommandResult(_format_missing_policy_decision(item, current_cfg))
548-
549:     async def _resolve_source(
550-         self, conn:aiosqlite.Connection, user_id: int, target: str
551-     ) -> Source | None:
552-         """Resolve a user-provided source identifier to a ``Source`` row.
553-
554:         Accepts numeric channel id, ``@username``, or channel title.
555:         ``@`` prefix is stripped before lookup.
556-         """
557-
558-
559-
560-
561-
562-
563-
564-         if src is not None:
565-             return src
566-         # Try by title (case-insensitive substring match).
567-         all_srcs = await db.list_account_sources(conn, user_id)
568-         target_lower = target.lower()
569-         for s in all_srcs:
570-             if s.title and target_lower in s.title.lower():
571-                 return s
572-         return None

```

26. assistant (2026-07-07T03:36:14.526Z)

Title-fragment resolution is supported. README's `/watch <id|@name|title-fragment>` is accurate.

Now let me check the README bootstrap invocation. README says `python -m tg_video_filter.bootstrap`. Let me verify that module is runnable, and check `.env.example` for `TG_API_ID/TG_API_HASH` naming.

27. user (2026-07-07T03:36:16.829Z)

```

.env.example
TG_API_ID=123456
TG_API_HASH=0123456789abcdef0123456789abcdef
---bootstrap main---
152:def main() -> int: # pragma: no cover - interactive entrypoint
171:if __name__ == "__main__":

```

28. assistant (2026-07-07T03:36:19.708Z)

Bootstrap and env vars confirmed. Now let me check the tables to verify the ADR 0011 status note and the design/plan/runbook docs. Let me look at the DB schema for table names and deprecated tables, plus check the design and runbook diffs.

29. user (2026-07-07T03:36:21.436Z)

```

43:CREATE TABLE IF NOT EXISTS accounts
51:CREATE TABLE IF NOT EXISTS telegram_sessions
60:CREATE TABLE IF NOT EXISTS channels
69:CREATE TABLE IF NOT EXISTS videos
86:CREATE TABLE IF NOT EXISTS filter_config
97:CREATE TABLE IF NOT EXISTS sources
110:CREATE TABLE IF NOT EXISTS content_items
133:CREATE TABLE IF NOT EXISTS policies
148:CREATE TABLE IF NOT EXISTS destinations
160:CREATE TABLE IF NOT EXISTS routing_rules
172:CREATE TABLE IF NOT EXISTS delivery_attempts
186:CREATE TABLE IF NOT EXISTS policy_decisions
224:_deprecated
227:_deprecated
375:_deprecated
381:_deprecated

```

443:_deprecated
444:_deprecated
516:_deprecated
528:_deprecated
658:_deprecated
669:_deprecated
677:_deprecated
678:_deprecated
679:_deprecated
687:_deprecated
689:_deprecated
700:_deprecated
703:_deprecated
725:_deprecated
745:_deprecated
760:_deprecated
783:CREATE TABLE IF NOT EXISTS policy_decisions
1585:_deprecated
1891:_deprecated

30. assistant (2026-07-07T03:36:24.904Z)

All tables confirmed: accounts, telegram_sessions, channels (legacy), videos (legacy), filter_config (legacy), sources, content_items, policies, destinations, routing_rules, delivery_attempts, policy_decisions. The ADR 0011 note listing sources/content_items/policies/destinations/routing_rules/delivery_attempts/policy_decisions is accurate; legacy channels/videos/filter_config are shims.

Now let me read the design, plan, and runbook diffs. These are large. Let me read them.

31. user (2026-07-07T03:36:26.474Z)

```
diff --git a/docs/design.md b/docs/design.md
index 45fc4aa..1c112c4 100644
--- a/docs/design.md
+++ b/docs/design.md
@@ -1,280 +1,233 @@
-# Design: Telegram Video Filter Userbot (v1)
-
-This document describes the runtime architecture, data model, control flow, and
-key technical contracts for v1. See [plan.md](./plan.md) for the phased build
-order and [adr/](./adr/) for the rationale behind the major decisions.
-
-## 1. Goals & non-goals
-
-###Goals (v1)**
-- Log in as a user account (Telethon MTPROTO userbot) and receive `NewMessage`
- events for every channel/group that account is a member of.
-- For each incoming **video** message, extract metadata **without downloading**
- the file, and apply a configurable set of filters.
-- **Forward** videos that match all filters to the user's Saved Messages in real
- time.
-- Support multiple users (<100), each with their own session, filter config, and
- seen-state, on a single host.
-- Be deployable as a single Docker container with a mounted data volume.
-
-###Non-goals (deferred to v2+)**
-- Bot API / public-channel auto-join / manual-add ingestion paths.
- *(Superseded for v2 by [ADR 0008](./adr/0008-bot-api-onboarding-front-door.md),
- which adds a Bot API front-door as the public onboarding path while keeping
- the MTPROTO userbot as opt-in power mode. Tracked in issue #25.)*
-- Selective channel scanning and per-source ? per-destination routing.
- *(Planned in [ADR 0009](./adr/0009-selective-channel-scanning.md);
- the current behaviour ? ingest every channel the account is a member of ?
- remains in effect until `channels.watched` lands.)*
```

```

-- Scheduled digest delivery and on-demand `/list` / `/search` commands.
-- ffmpeg-based deep inspection (`/inspect`).
-- Codec / keyword / MIME filters.
-- Horizontal scaling / sharding across hosts.
-
-## 2. Component overview
-
-...
-????????????????????????????????????????????????????????????
-? Manager (asyncio supervisor)   src/tg_video_filter/manager ?
-?                                ?
-?   for each enabled user in DB:   ?
-?     build_client(user) ??? TelegramClient (Telethon)      ?
-?       register events.NewMessage(incoming=True) handler    ?
-?       client.run_until_disconnected() (one asyncio.Task)   ?
-?                                ?
-?   SIGTERM/SIGINT ? cancel all tasks, disconnect clients    ?
-????????????????????????????????????????????????????????????
-?                                ?
-?                                ?
-????????????????????????????????????????????????????????????
-? Ingestion (telethon_client) ? ? Delivery (delivery.py) ?
-?                                ? ?
-?   on NewMessage:              ? ? forward_to_target(client,?
-?     if message.video is None: ? ? message, target):        ?
-?       ignore                   ? ? target = cfg.delivery_ ?
-?     extract VideoMeta          ? ? target ('me' /           ?
-?     dedup 3-tier (ch,msg,doc,fp) ? ? '@user' / chat id)    ?
-?     FilterEngine.evaluate()    ? ? try forward_messages    ?
-?     if matched: deliver        ???? (target, message)      ?
-????????????????????????????????????????????????????????????
-?                                ? except FloodWaitError: ?
-?                                ? sleep+jitter, retry ?
-?                                ?
-?                                ?
-?                                ?
-????????????????????????????????????????????????????????????
-? FilterEngine (filters/) ?
-?   build from FilterConfig ? [DurationFilter,
-?                               ResolutionFilter, SizeFilter] ?
-?   evaluate(meta) ? FilterResult(matched, reasons[]) ?
-?   semantics: AND (all must match) ?
-????????????????????????????????????????????????????????????
-?                                ?
-?                                ?
-????????????????????????????????????????????????????????????
-? SQLite (aiosqlite) db.py ?
-?   tables: users, channels, videos, filter_config ?
-?   repos: get/create user, upsert video (dedup), ?
-?           load/save filter_config ?
-????????????????????????????????????????????????????????????
+# Design: Telegram Video Filter Userbot
+
+This document describes the current runtime architecture and data contracts.
+Decision rationale lives in [docs/adr/](./adr/); operational steps live in
+[runbook.md](./runbook.md).
+
+## 1. Goals
+
+- Run one Telethon MTProto userbot session per enabled Telegram account.
+- Discover Telegram channels/groups as provider-neutral `sources`.
+- Ingest live videos only from sources explicitly marked `watched=true`.
+- Extract message metadata and captions without downloading media on the normal
+  path.
+- Apply metadata filters and the ADR 0012 policy pipeline.
+- Deliver accepted content to a configured Telegram destination.

```

```

+- Persist enough state to deduplicate, retry/inspect delivery state, and explain
+ decisions with `/why`.
+
+## 2. Non-goals
+
+- Bot API onboarding/front-door commands. ADR 0008 defines the shape, but it is
+ not implemented.
+- Public-channel auto-join or manual `/add @channel` ingestion.
+- Scheduled digest delivery or search/list commands.
+- Production deep-inspection fetch/download work. The policy stage and budget
+ boundary exist; no fetcher is wired by default.
+- Horizontal scaling across multiple hosts.
+
+## 3. Runtime Components
+
+```text
+python -m tg_video_filter
+ |
+ v
+manager.run()
+ |
+ +-- loads runnable telegram_sessions
+ +-- starts one Telethon client task per session
+ |
+ +-- live NewMessage handler
+ |   source watch gate -> video metadata -> dedup -> FilterEngine
+ |   -> PolicyStage pipeline -> content_items/policy_decisions
+ |   -> delivery.deliver()
+ |
+ +-- Saved Messages command handlers
+ |   /show, /set, /channels, /watch, /unwatch, /watchlist, /why
+ |   /backfill, /stop
+ ...
+
+## 3. Data model
+
+-
+-All tables live in a single SQLite file (default `data/app.db`).
+-
+### `users`
+-| column | type | notes |
+-|---|---|---|
+-| `id` | INTEGER PK | Telegram user id (the account the bot logs in as) |
+-| `phone` | TEXT | E.164 phone, used only for the bootstrap session |
+-| `enabled` | INTEGER (0/1) | If 0, manager skips this user |
+-| `created_at` | TEXT (ISO8601) | |
+-
+### `channels`
+-Row per (user, channel) the user is a member of. Populated lazily as messages
+-arrive; primarily informational in v1.
+-
+-| column | type | notes |
+-|---|---|---|
+-| `id` | INTEGER PK | surrogate |
+-| `user_id` | INTEGER FK ? users.id | |
+-| `channel_id` | INTEGER | Telegram channel/chat id |
+-| `title` | TEXT | last seen title |
+-| `seen_first_at` | TEXT (ISO8601) | |
+-
+-Unique index on `(user_id, channel_id)`.
+-
+### `videos`
+-Seen-state + metadata log. Drives dedup and future `/search`.
+-
+-| column | type | notes |
+-|---|---|---|

```

```

-| `id` | INTEGER PK | surrogate |
-| `user_id` | INTEGER FK ? users.id | |
-| `channel_id` | INTEGER | origin channel |
-| `message_id` | INTEGER | Telegram message id within the channel |
-| `document_id` | INTEGER | Telegram `document.id` ? reused across forwards; **Tier-1 content
dedup** (nullable; ADR 0006) |
-| `duration_s` | INTEGER | from `DocumentAttributeVideo.duration` |
-| `width` | INTEGER | from `DocumentAttributeVideo.w` |
-| `height` | INTEGER | from `DocumentAttributeVideo.h` |
-| `size_bytes` | INTEGER | from `document.size` |
-| `mime_type` | TEXT | from `document.mime_type` |
-| `matched` | INTEGER (0/1) | did it pass filters |
-| `forwarded` | INTEGER (0/1) | did the forward succeed |
-| `seen_at` | TEXT (ISO8601) | |
-
-Unique index on `(user_id, channel_id, message_id)` ? the Tier-0 dedup key.
-
-A partial unique index on `(user_id, document_id)` WHERE document_id IS NOT NULL`
-enforces **Tier-1 content dedup** (same Telegram file forwarded across channels).
-**Tier-2 content dedup** uses a metadata fingerprint
-`(size_bytes, duration_s, height, width)` lookup with no DB constraint (it is
-heuristic, not cryptographic). See [ADR 0006](./adr/0006-content-dedup.md).
-
-### `filter_config`
-One row per user (v1). Stored as JSON blob for forward-compat as new filters land.
-
-| column | type | notes |
-|---|---|---|
-| `user_id` | INTEGER PK FK ? users.id | |
-| `config_json` | TEXT | serialized `FilterConfig` |
-
-Default v1 config (v1.1 adds ``delivery_mode``):
+The process uses one SQLite database. Telethon session files live under
+`SESSIONS_DIR` (`data/sessions` locally, `/data/sessions` in Docker).
+
+### 4. Identity and Sessions
+
+ADR 0010 split identity from MTPROTO sessions:
+
+- `accounts` owns config, sources, content, policies, destinations, and routes.
+- `telegram_sessions` is optional MTPROTO state for accounts that run as
+  userbots.
+- Legacy `users` APIs still work as shims over `accounts` +
+  `telegram_sessions`.
+
+This lets a future Bot API front door create accounts without also creating a
+Telethon session.
+
+### 5. Source Scanning
+
+Live scanning is opt-in:
+
+1. A first live message from an unknown Telegram channel/group creates a
+  `sources` row with `watched=false`.
+2. The live handler returns immediately for unwatched sources.
+3. Users list and toggle sources from Saved Messages:
+  - `/channels`
+  - `/watch <id|@name|title-fragment>`
+  - `/unwatch <id|@name|title-fragment>`
+  - `/watchlist`
+
+Backfill is different: `/backfill` iterates historical dialogs directly and can
+scan broadly. It should be treated as an operational tool, not the live watch
+gate.
+

```

```

+## 6. Data Model
+
+Important tables:
+
+| Table | Purpose |
+|---|---|
+| `accounts` | Provider-neutral owner identity. |
+| `telegram_sessions` | Optional Telethon session metadata per account. |
+| `sources` | Provider-neutral ingest sources, including `watched`. |
+| `content_items` | Normalized content records and dedup keys. |
+| `policies` | JSON policy config; one default policy per account. |
+| `destinations` | Delivery targets such as `"me"`, `@channel`, or chat id. |
+| `routing_rules` | Source -> policy -> destination mapping skeleton. |
+| `delivery_attempts` | Per-content/per-destination delivery state. |
+| `policy_decisions` | Persisted ADR 0012 pass/fail decision and stage trace. |
+
+Legacy tables/models (`users`, `channels`, `videos`, `filter_config`,
+`VideoRecord`, `FilterConfig`) are still supported through compatibility shims.
+New code should prefer `Account`, `TelegramSession`, `Source`, `ContentItem`,
+`Policy`, `Destination`, `DeliveryAttempt`, and `PolicyDecision`.
+
+#### Dedup
+
+`content_items.dedup_key` is provider-prefixed and unique per account. Telegram
+dedup is computed cheapest-first:
+
+1. Same source/message instance.
+2. Same Telegram `document.id`.
+3. Same metadata fingerprint `(size_bytes, duration_s, height, width)`.
+
+## 7. Filter and Policy Evaluation
+
+Evaluation is two-layered:
+
+1. `FilterEngine` applies the flat metadata `FilterConfig` rules:
+  - min/max duration
+  - minimum height
+  - maximum size
+  - GIF, round-video, and nosound exclusions
+2. If metadata filters pass and a `Source` exists, the ADR 0012 policy pipeline
+  evaluates:
+  - `TopicPreferenceStage`
+  - `UrlReputationStage`
+  - `CaptionSafetyStage`
+  - `DeepInspectionStage`
+
+The policy stages return ordered `StageResult` values. A failed stage
+short-circuits the pipeline. F6 persists the final pass/fail and full stage
+trace in `policy_decisions`. F7 exposes it through `/why <content_item_id>`.
+
+#### Policy Config Shape
+
+Flat filter fields still live in `FilterConfig`, but `policies.config_json` can
+also hold future policy keys. The DB shims merge flat updates into the existing
+JSON object so `/set` commands do not erase unknown policy keys.
+
+Relevant policy keys today:
+
+```json
+{
+  - "min_duration_s": 5,
+  - "max_duration_s": 7200,
+  - "target_height": 1080,
+  - "max_size_mb": 2048,
+  - "delivery_mode": "forward"
+}

```

```
+ "topics": {
+   "include": ["ai"],
+   "exclude": ["crypto"],
+   "mode": "any"
+ },
+ "security_checks": {
+   "blocked_domains": ["bad.example"],
+   "unsafe_caption_terms": ["malware"],
+   "unsafe_caption_patterns": ["free\\s+money"]
+ },
+ "deep_inspect": false
+ }
+ ````
```

4. Delivery contract (v1.1)

+There are no user-facing `/set` commands for these policy keys yet; edit
+`policies.config.json` deliberately if you need them.

```
-``python
```

```
-class DeliveryMode(str, Enum):
```

```
-    FORWARD = "forward" # forward the video itself (default)
-    LINK = "link"        # send a t.me link + metadata summary
-    BOTH = "both"        # forward + send link
```

```
### 8. Delivery
```

```
-async def deliver(client, message, conn, video_id, mode, *, link_text=None) -> bool
-````
```

+`FilterConfig.delivery\_mode` controls how accepted content is delivered:

-The `deliver` dispatcher in `delivery.py` routes to:

```
-- `FORWARD` ? `client.forward_messages("me", message)`
-- `LINK` ? `client.send_message("me", link_text)` where `link_text` is
- built by the pure `build_link_text(...)` function
-- `BOTH` ? forward + send link
```

```
+| Mode | Behavior |
```

```
+|---|---|
```

```
+| `forward` | Forward the Telegram message to the target. |
```

```
+| `link` | Send a `t.me` link plus metadata summary. Falls back to forward if no link text is
available. |
```

```
+| `both` | Forward and send the link summary. |
```

-Link message format:

```
-````
-? 1080p · 5min · 480MB
-#ChannelName
-https://t.me/c/1234567890/42
-````
```

+`FilterConfig.delivery\_target` is passed to Telethon as the destination entity:

-If `link\_text` is `None` in LINK/BOTH mode (e.g. `message.link` was

-unavailable), falls back to FORWARD so no match is silently dropped. See

-[ADR 0004](./adr/0004-pluggable-delivery-modes.md).

+- `me` for Saved Messages

+- `@username`

+ numeric chat id string such as `-1001234567890`

5. Filter engine contract

+Delivery writes `delivery\_attempts`; policy rejection is not a delivery failure
+and is stored in `policy\_decisions` instead.

```
-``python
```

```
-class VideoMeta: # extracted from the Telethon message
```

```
-    duration_s: int | None
-    width: int | None
-    height: int | None
```

```

-     size_bytes: int | None
-     mime_type: str | None
+## 9. Commands

-class FilterResult:
-     matched: bool
-     reasons: list[str]          # human-readable, one per evaluated rule
+Saved Messages commands are handled by provider-independent `CommandService`
+plus a thin Telethon adapter.

-class Filter(ABC):                # filters/base.py
-     @abstractmethod
-     def evaluate(self, meta: VideoMeta) -> FilterResult: ...
+Config:

-class FilterEngine:                # filters/__init__.py
-     def __init__(self, cfg: FilterConfig): ...
-     def evaluate(self, meta: VideoMeta) -> FilterResult: ...    # AND of all rules
- ````
+- `/show config`
+- `/show commands`
+- `/set size <N><unit>`
+- `/set length <N><unit>` or `/set max length <N><unit>`
+- `/set min length <N><unit>`
+- `/set min resolution <N>p` or `/set min res <N>p`
+- `/set exclude gif|round|nosound on|off`

-Semantics: AND ? a video is forwarded only if every enabled rule passes.
-A rule that cannot be evaluated (e.g. `duration is None`) is treated as
-not matched, with a reason like `"duration unknown"`.
-
-### v1 rules
-| rule | pass condition | fail reason examples |
-|---|---|---|
-| `DurationFilter` | `min_duration_s <= duration <= max_duration_s` | `"duration 3s < min 5s",`
-| | | | |
-| | | | |
-| `ResolutionFilter` | `height >= target_height` | `"height 720 < target 1080" |
-| `SizeFilter` | `size_bytes <= max_size_mb * 1_048_576` | `"size 3000MiB > max 2048MiB" |
-| | | | |
-
-## 6. Control flow: incoming video
-
-1. Telethon fires `events.NewMessage(incoming=True)`.
-2. Handler checks `event.message.video is not None`; if None, return.
-3. Build `VideoMeta` from the message document + `DocumentAttributeVideo`.
-4. Run the three-tier dedup check (cheapest first; see ADR 0006):
-   - Tier 0: `(user_id, channel_id, message_id)` ? same message instance.
-   - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across
-     channels (DB-enforced via partial unique index).
-   - Tier 2: `(size_bytes, duration_s, height, width)` fingerprint ?
-     re-upload of an identical file with a new `document.id`.
-   - If any tier matches, return (already processed).
-5. Build `FilterEngine` from the user's `filter_config` (cached for the session).
-6. `result = engine.evaluate(meta)`.
-7. Insert a `videos` row with `match=result.matched`.
-8. If `result.matched`: call `delivery.deliver(client, message, conn, id,
-   mode, link_text=..., target=cfg.delivery_target)`. `delivery_target` is a
-   Telethon entity: `"me"` (Saved Messages, default), `"@username"`, or a
-   numeric chat id string (see ADR 0005). On success set `forwarded=1`. On
-   `FloodWaitError`, sleep and retry once; on second failure leave
-   `forwarded=0` and log.
-
-## 7. Control flow: manager lifecycle
+Source scanning:
-
- ````

```

```

-main()
- ?? load Settings (env)
- ?? await db.init(db_path)          # create tables if missing
- ?? users = await db.list_enabled_users()
- ?? for u in users:
- ?     task = asyncio.create_task(run_user(u))
- ?? install SIGTERM/SIGINT handlers ? cancel all tasks
- ?? await asyncio.gather(*tasks)
-
-run_user(u)
- ?? client = await build_client(u)    # SQLiteSession at data/sessions/user_{id}.session
- ?? engine = FilterEngine(await db.load_filter_config(u.id))
- ?? register NewMessage handler bound to (client, u, engine)
- ?? await client.run_until_disconnected()
- ``
+- `/channels`
+- `/watch <id|@name|title-fragment>`
+- `/unwatch <id|@name|title-fragment>`
+- `/watchlist`
+
+Backfill and explainability:
+
+- `/backfill [N][h|d|w]`
+- `/stop`
+- `/why <content_item_id>`
+
+`/why` is account-scoped before reading decisions, escapes untrusted Markdown
+values, breaks raw autolinks, and caps replies below Telegram's message limit.
+
+## 10. Observability
+
+The app logs to stdout. Important events include:
+
+- `manager_started`
+- `client_connected`
+- `video_seen`
+- `video_forwarded`
+- `delivery_record_failed`
+- `flood_wait`
+- `config_updated`
+- policy-stage warnings for invalid config or failed deep inspection
+- `client_disconnected`
+
+There is no metrics endpoint. Docker restart policy and logs are the current
+operational surface.
+
+## 11. Security Notes
+
+## 8. Security & operational notes
+
+- **Secrets**: `TG_API_ID`, `TG_API_HASH`, and session files (`*.session`) are
+  sensitive. `.gitignore` excludes `*.session` and `.env`. In Docker, the
+  `data/` volume holds both the SQLite DB and session files ? back it up, treat
+  it as a credential.
+- **Session rotation**: see [`runbook.md`](./runbook.md) `"$Rotate a user session"`.
+- **Flood control**: forwarding too fast triggers `FloodWaitError`. v1 sleeps the
+  requested duration + small jitter and retries once. If your account gets
+  flagged, slow down or reduce the number of channels.
+- **ToS**: a userbot automates a *user* account. Keep volume reasonable, don't
+  spam, don't scrape at scale. The owner is responsible for their account.
+
+## 9. Observability (v1 minimum)
+
+- Structured logging to stdout (JSON Lines recommended). Key events:
+- `manager_started` `{users: N}`

```

```
-- `client_connected` `{user_id}`
-- `video_seen` `{user_id, channel_id, message_id, matched, reasons}`
-- `video_forwarded` `{user_id, channel_id, message_id}`
-- `flood_wait` `{user_id, seconds}`
-- `client_disconnected` `{user_id, reason}`
-
-A `/health` or simple process-level liveness check is a v2 concern; v1 relies on
-Docker restart policy.
+- Treat `.env`, `TG_API_HASH`, and `*.session` files as credentials.
+- `/why` only exposes content belonging to the requesting account.
+- Untrusted captions, URLs, reasons, and metadata are escaped in `/why` output.
+- Backfill can generate significant Telegram traffic; run it intentionally.
+- Userbot automation can put the Telegram account at risk if abused.
```

32. assistant (2026-07-07T03:36:30.196Z)

Now let me verify several concrete design.md claims. Let me check the log event names (`video\_seen`, `video\_forwarded`, `delivery\_record\_failed`, `config\_updated`, `client\_disconnected`), the delivery_target behavior (link fallback to forward), and the `/why` markdown-escape claims. Let me check log events and delivery.

33. user (2026-07-07T03:36:31.854Z)

(Bash completed with no output)

34. user (2026-07-07T03:36:35.439Z)

```
1 client_connected
1 client_disconnected
1 config_updated
2 flood_wait
1 manager_started
1 video_forwarded
2 video_seen
```

35. assistant (2026-07-07T03:36:37.986Z)

Most log events confirmed. `delivery\_record\_failed` was not in that grep. Let me check for it specifically and confirm the design doc's log event list is accurate.

36. user (2026-07-07T03:36:39.554Z)

NOT FOUND

37. assistant (2026-07-07T03:36:42.461Z)

`delivery\_record\_failed` doesn't exist. This is a minor doc inaccuracy in the design doc's observability list, but log-event names are illustrative and low-impact. Let me check what the actual delivery-related log events are, to gauge whether this is misleading.

38. user (2026-07-07T03:36:44.942Z)

(Bash completed with no output)

39. user (2026-07-07T03:36:48.573Z)

```
116:         log.exception("delivery_retry_failed video=%s", video_id)
119:         log.exception("delivery_failed video=%s", video_id)
266:         log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
268:         log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
```

40. assistant (2026-07-07T03:36:52.650Z)


```

+## Current State
+
+The main Telegram userbot workflow is implemented:
+
+1. Bootstrap Telegram user sessions with `python -m tg_video_filter.bootstrap`.
+2. Run one Telethon client per enabled session with `python -m tg_video_filter`.
+3. Discover provider-neutral `sources`.
+4. Opt in live ingestion with `/watch`.
+5. Deduplicate content into `content_items`.
+6. Evaluate metadata filters and ADR 0012 policy stages.
+7. Persist policy decisions and delivery attempts.
+8. Deliver accepted content by forward, link, or both.
+9. Explain decisions with `/why <content_item_id>`.
+
+## Completed Tracks
+
+| Area | Status | Notes |
+|---|---|---|
+| Core scaffold | Complete | Python package, Docker, settings, tests, lint, coverage gate. |
+| Data layer | Complete | SQLite schema, migrations through `user_version=5`, provider-neutral repos and legacy shims. |
+| Filter engine | Complete | Duration, resolution, size, GIF, round-video, and nosound filters. |
+| Telethon ingestion | Complete | Metadata extraction, source gate, dedup, policy evaluation, persistence, delivery. |
+| Delivery | Complete | `forward`, `link`, `both`; configurable target; delivery attempts. |
+| Bootstrap | Complete | Interactive login creates session file and default account/session/config rows. |
+| Backfill | Complete | `/backfill [N][h|d|w]` plus `/stop`; broad historical scan. |
+| Inline commands | Complete | `/show`, `/set`, source watch commands, `/why`. |
+| Content dedup | Complete | Message instance, Telegram `document.id`, metadata fingerprint. |
+| Selective scanning | Complete | `sources.watched` and Saved Messages watchlist commands. |
+| Identity/session split | Complete | `accounts` and `telegram_sessions`, with legacy `users` shim. |
+| Provider-neutral schema | Complete | `sources`, `content_items`, `policies`, `destinations`, `routing_rules`, `delivery_attempts`. |
+| Policy pipeline | Complete | ADR 0012 F1-F7: engine, topics, caption safety, URL reputation, deep-inspection boundary, audit table, `/why`. |
+
+## ADR 0012 Phase Closure
+
+| Phase | Issue | PR | Result |
+|---|---|---|---|
+| F1 - Policy engine prototype + config persistence | #43 | #50 | Merged |
+| F2 - Topic preferences + caption capture | #44 | #51 | Merged |
+| F3 - Caption safety stage | #45 | #52 | Merged |
+| F4 - URL/domain reputation stage | #46 | #53 | Merged |
+| F5 - Deep inspection fetcher + budget boundary | #47 | #54 | Merged |
+| F6 - Policy decisions audit table | #48 | #55 | Merged |
+| F7 - `/why` explainability command | #49 | #57 | Merged |
+
+Umbrella issue #42 is complete. Policy-decision retention cleanup remains open as follow-up #56.
+
+## Current Operational Commands
+
+Config:
+
+- `/show config`
+- `/show commands`
+- `/set size <N><unit>`
+- `/set length <N><unit>` or `/set max length <N><unit>`
+- `/set min length <N><unit>`
+- `/set min resolution <N>p` or `/set min res <N>p`
+- `/set exclude gif|round|nosound on|off`

```

```

+
+Source scanning:
+
+- `/channels`
+- `/watch <id|@name|title-fragment>`
+- `/unwatch <id|@name|title-fragment>`
+- `/watchlist`
+
+Backfill and explainability:
+
+- `/backfill [N][h|d|w]`
+- `/stop`
+- `/why <content_item_id>`
+
+### Deferred Work
+
+- Bot API front door and onboarding commands from ADR 0008.
+- Public-channel auto-join/manual-add ingestion.
+- Scheduled digest delivery.
+- On-demand search/list commands.
+- Production deep-inspection fetcher/downloader.
+- Policy-decision retention cleanup (#56).
+- Horizontal scaling or multi-host coordination.
+
+### Verification Baseline
+
+Use the repo-local virtualenv on Windows:
+
+```powershell
+.\.venv\Scripts\python.exe -m ruff check src tests
+.\.venv\Scripts\python.exe -m pytest
+```

-## Project structure
-```
-??? README.md
-??? .gitignore
-??? .env.example                # TG_API_ID, TG_API_HASH, DB_PATH, ADMIN_USER_IDS
-??? pyproject.toml              # deps: telethon, aiosqlite, pydantic, pydantic-settings
-??? .pre-commit-config.yaml     # ruff, ruff-format
-??? docker-compose.yml         # service: app (mounts ./data for sessions+db)
-??? Dockerfile                 # python:3.11-slim, install deps, run manager
-??? docs/
-?   ??? plan.md                 # this file
-?   ??? design.md               # architecture detail
-?   ??? adr/                     # Architecture Decision Records
-?   ?   ??? 0001-userbot-over-botapi.md
-?   ?   ??? 0002-sqlite-over-postgres.md
-?   ?   ??? 0003-metadata-first-probing.md
-?   ??? runbook.md              # deploy, rotate session, handle FloodWait
-??? src/tg_video_filter/
-?   ??? __init__.py
-?   ??? __main__.py             # `python -m tg_video_filter` ? run manager
-?   ??? config.py               # pydantic-settings, env loading
-?   ??? db.py                   # aiosqlite schema + migrations + repos
-?   ??? models.py               # User, Channel, Video, FilterConfig pydantic models
-?   ??? filters/
-?   ?   ??? __init__.py         # FilterEngine registry
-?   ?   ??? base.py             # Filter ABC: match(metadata) -> bool, reason str
-?   ?   ??? duration.py
-?   ?   ??? resolution.py
-?   ?   ??? size.py
-?   ??? telethon_client.py      # per-user client factory, event wiring
-?   ??? manager.py              # asyncio supervisor: start/stop per-user clients
-?   ??? delivery.py             # forward-to-saved-messages (v1); digest/search in v2

```

```

-??? data/                                # gitignored: sessions/*.session, app.db
-??? tests/
-    ??? conftest.py
-    ??? test_filters_duration.py
-    ??? test_filters_resolution.py
-    ??? test_filters_size.py
-    ??? test_filter_engine.py
-...
+The suite currently has broad unit coverage around pure logic and repository
+behavior. Live Telethon glue and interactive prompts are intentionally marked
+`pragma: no cover`.
+
+### Original v1 Scope, Now Superseded
+
+The original v1 plan was a lean userbot that watched every subscribed channel
+and forwarded matching videos to Saved Messages. That was useful early, but the
+current implementation has moved beyond it:
+
+- live ingestion is opt-in per source;
+- provider-neutral schema is the canonical model;
+- delivery is per target/mode with delivery attempts;
+- policy decisions are explainable and persisted.

-## Steps (phased)
-
-### Phase 0 ? Scaffolding (no runtime logic) ? DONE (PR #2)
-1. ? `git init`, create `.gitignore` (Python, `data/`, `*.session`, `.env`), `README.md`.
-2. ? `pyproject.toml` with deps: `telethon`, `aiosqlite`, `pydantic`, `pydantic-settings`,
`python-dotenv`, `structlog`; dev deps: `pytest`, `pytest-asyncio`, `pytest-cov`, `ruff`, `pre-
commit`. Coverage floor `fail_under = 80`.
-3. ? `.pre-commit-config.yaml` ? ruff check + format (+ pre-commit-hooks hygiene).
-4. ? `Dockerfile` (python:3.11-slim base) + `docker-compose.yml` (mount `./data`).
-5. ? `.env.example` documenting `TG_API_ID`, `TG_API_HASH`, `DB_PATH`, `SESSIONS_DIR`,
`ADMIN_USER_IDS`, `LOG_LEVEL`.
-6. ? `src/tg_video_filter/__init__.py`, `config.py` (pydantic-settings + `NoDecode` CSV
parsing), `__main__.py` placeholder; `tests/conftest.py`, `tests/test_config.py`,
`tests/test_main.py`.
-   - Verification: `pytest` 3 passed @ **94.6% coverage**; `ruff check .` + `ruff format
--check .` clean; `python -m tg_video_filter` runs end-to-end.
-   - Note: pydantic-settings v2.14 requires `Annotated[list[int], NoDecode]` to keep CSV env
parsing (`ADMIN_USER_IDS=1,2,3`) out of the JSON decoder.
-
-### Phase 1 ? Core data layer ? DONE (PR #3)
-7. ? `src/tg_video_filter/config.py` ? done in Phase 0 (pydantic-settings `Settings`).
-8. ? `src/tg_video_filter/models.py` ? pydantic models: `User`, `Channel`, `VideoRecord`,
`FilterConfig` (+ `DEFAULT_FILTER_CONFIG`, `dedup_key` helper). Frozen models; nullable filter
bounds to disable a rule.
-9. ? `src/tg_video_filter/db.py` ? aiosqlite schema DDL (users, channels, videos,
filter_config), `init_db()`, repository functions (`upsert_user`, `get_user`,
`list_enabled_users`, `upsert_channel`, `get_video`, `insert_video`, `mark_forwarded`,
`load_filter_config`, `save_filter_config`, `list_videos_for_user`). Connection injected so
tests use in-memory SQLite.
-10. ? `tests/test_db.py` ? 16 round-trip tests incl. dedup-key IntegrityError, None-bounds JSON
round trip, enabled-user filter, DESC ordering.
-   - Verification: `pytest` 19 passed @ **98% coverage**; `ruff check .` + `ruff format
--check .` clean.
-
-### Phase 2 ? Filter engine (pure, fully unit-testable) ? DONE (PR #4)
-11. ? `filters/base.py` ? `VideoMeta` (pydantic), `FilterResult` (dataclass with `pass_`/`fail`
helpers), `Filter` ABC.
-12. ? `filters/duration.py`, `resolution.py`, `size.py` ? concrete filters. Each treats an
unknown metadata field as a non-match with a descriptive reason; a `None` bound disables the
rule.
-13. ? `filters/__init__.py` ? `FilterEngine`: builds the rule list from `FilterConfig`
(omitting disabled rules), evaluates a `VideoMeta` against **all** rules with AND semantics,

```

```

collects every reason.
- - Verification: `pytest` 58 passed @ **99% coverage** (every filter module 100%); `ruff
check .` + `ruff format --check .` clean.
- - 36 filter tests: table-driven boundary cases (inclusive bounds, None-bounds, unknown
fields), AND-semantics aggregation, partial-config scenarios, vacuous-match guard.
-
-### Phase 3 ? Telethon ingestion ? DONE (PR #5)
-13. ? `telethon_client.py` ? `build_client(user, settings)` per-user Telethon client factory
(file-backed SQLiteSession at `data/sessions/user_{id}.session`);
`make_new_message_handler(...)` wires `events.NewMessage` ? dedup ? filter ? insert ? forward.
-14. ? `extract_video_meta(message)` ? **pure** metadata extraction: inspects `message.media` (a
`Document`) directly for `size`/`mime_type`, and `DocumentAttributeVideo` for `duration`
(float?rounded int)/`w`/`h`. Returns `None` for non-videos. Returns `VideoMeta` with `None`
fields for partial data. (Note: uses `message.media`, not the `.video` convenience property,
which is only present on Telethon's runtime *custom* Message wrapper ? not the raw TL
`Message`.)
-15. ? Dedup: handler queries `db.get_video(user, channel, msg)`; skips if seen. `_peer_id`
collapses `PeerChannel`/`PeerChat`/`PeerUser` to an int.
-16. ? `delivery.forward_to_saved` pulled forward (was Phase 4) since the handler imports it:
wraps `client.forward_messages('me', msg)` with `FloodWaitError` handling (sleep `seconds` +
`jitter`, retry once), marks `forwarded=1` on success.
- - Verification: `pytest` 77 passed @ **89% coverage** (`delivery.py` 100%, `filters/`
100%, `db.py` 99%, `models.py` 100%); `ruff check .` + `ruff format --check .` clean.
- - 19 new tests: 14 for `extract_video_meta` (full metadata, float rounding, zero?None
normalization, non-video/photo/text?None, non-video-attr Document?None, image-size attr ignored)
+ `_peer_id` helper; 5 for `forward_to_saved` (success, FloodWait?retry success, FloodWait?retry
fail, generic exception, seconds-attr guard).
- - **Uncovered (57%): `telethon_client.py` handler/factory glue** ? async Telethon plumbing
requiring a live connection; documented as intentional.
-
-### Phase 4 ? Delivery + manager ? DONE (PR #6)
-17. ? `delivery.py` ? done in Phase 3 (`forward_to_saved` + `FloodWaitError` handling, 100%
coverage).
-18. ? `manager.py` ? `run(settings, client_runner)` asyncio supervisor: opens one shared DB
connection, loads enabled users, spawns one `asyncio.Task` per user,
`gather(return_exceptions=True)` so one crash doesn't tear down others, installs SIGINT/SIGTERM
handlers. `default_run_user` (the production runner) builds the Telethon client, checks
`is_user_authorized()`, builds the `FilterEngine`, registers `events.NewMessage`, and
`run_until_disconnected()`. **Client runner is injected** so the supervisor logic is unit-
testable without a live connection.
-19. ? `__main__.py` ? real entrypoint: load Settings, configure logging,
`asyncio.run(run(settings, functools.partial(default_run_user, settings=settings)))`, catch
`KeyboardInterrupt`.
- - Verification: `pytest` 82 passed @ **84% coverage** (`manager.py` 67%, `__main__.py`
85%); `ruff check .` + `ruff format --check .` clean; `python -m tg_video_filter` runs end-to-
end (logs `manager_started users=0`, exits 0).
- - 6 new tests: no-users?0, N-users?N-tasks, one-crash-isolated, cancellation-propagates,
entrypoint importable + wires production runner.
- - **Uncovered**: `default_run_user` + signal handlers (need a live Telegram session / POSIX
signals; intentional).
-
-### Phase 5 ? First-run / onboarding ? DONE (PR #7)
-20. ? `src/tg_video_filter/bootstrap.py` ? interactive CLI (`python -m
tg_video_filter.bootstrap`) prompts phone/code/2FA via injectable callables, runs Telethon
`client.start()`, reads `get_me().id`, renames the session file to `user_{id}.session`, then
`persist_user()` inserts the `users` row + default `FilterConfig`. The DB persistence
(`persist_user`) is separated from the interactive login so it's unit-tested.
-21. ? Documented in `docs/runbook.md` ($3 onboard, plus cryptg note for prod crypto perf).
- - Verification: `pytest` 86 passed @ **78% coverage** (`bootstrap.persist_user` covered;
`bootstrap()`/`main()` interactive glue intentionally uncovered); `ruff check .` + `ruff format
--check .` clean; `python -m tg_video_filter.bootstrap` runs and reaches the login prompt.
- - Coverage gate temporarily lowered to `fail_under = 70` for the development phase
(interactive entrypoints can't be unit-tested); restores to 80 in Phase 6.
-
-### Phase 6 ? Tests + lint gate (final polish) ? DONE (PR #8)

```

```

-22. ? Marked all interactive / Telethon-glue entrypoints `# pragma: no cover`:
-   - `bootstrap.py`: `bootstrap()`, `main()`, the three `_default_*_prompt` stdin helpers.
-   - `__main__.py`: `_configure_logging()`, `main()`.
-   - `manager.py`: `default_run_user()`, `_install_signal_handlers()`.
-   - `telethon_client.py`: `build_client()`, `on_new_message()`, `_channel_title()`.
-23. ? Restored coverage gate `fail_under = 80` (was temporarily 70 during Phases 5?6).
-24. ? Final verification gate green; README marked v1 complete.
-   - **Final coverage: 98.6%** (every module 93%+; `filters/*`, `db.py`, `models.py`,
`delivery.py`, `__main__.py`, `manager.py`, `bootstrap.py` all 100%).
-
----
-
-## v1 sign-off ?
-
-All 8 PRs merged to `main`. The application is end-to-end usable:
-
-1. `cp .env.example .env` ? fill `TG_API_ID` / `TG_API_HASH` from my.telegram.org.
-2. `docker compose up -d --build` (or run from source).
-3. `python -m tg_video_filter.bootstrap` ? interactive login, creates session + user row.
-4. Restart ? manager watches all the account's channels, filters videos by
duration/resolution/size, forwards matches to Saved Messages.
-
-| Metric | Value |
-|---|---|
-| Tests | 86 passed |
-| Coverage | 98.6% (gate 80%) |
-| Lint | ruff clean |
-| Entry points verified | `python -m tg_video_filter` + `python -m tg_video_filter.bootstrap` |
-
-### Deferred to v2
-- Public-channel `/add @channel` join, manual-add ingestion path.
-- Scheduled digest delivery + on-demand `/list` `/search`.
-- ffprobe `/inspect` deep inspection.
-- Codec / keyword / MIME filters.
-
----
-
-## v1.1 ? Link-share delivery mode (in progress)
-
-### Phase 2a/2b ? DeliveryMode + link message + dispatcher ? DONE (PR #10)
-- **`models.py`**: added `DeliveryMode` enum (`FORWARD` | `LINK` | `BOTH`) and `delivery_mode`
field to `FilterConfig`. Backwards-compatible (old configs default to `FORWARD`).
-- **`delivery.py`**: refactored into a `deliver(client, msg, conn, video_id, mode,
link_text=...)` dispatcher. Added pure `build_link_text(...)` for the metadata-summary format
(`? 1080p · 5min · 480MB / #Channel / t.me link`). LINK mode with no link falls back to FORWARD
so no match is dropped.
-- **`telethon_client.py`**: handler now calls `deliver()` with the user's configured mode,
building link text from `message.link` + extracted metadata.
-- **`manager.py`**: passes `cfg.delivery_mode` through to the handler.
-- **Tests**: 104 passed @ 97.2% coverage; ruff clean. New tests: `build_link_text` formatting
(full/none/partial metadata), duration+size formatters (parametrized), `deliver` dispatcher
(forward/link/both/fallback/flood-retry/db-marking), FilterConfig delivery_mode JSON round-trip,
old-config-without-field defaults to FORWARD.
-- **Docs**: ADR 0004 (pluggable delivery modes), design.md §4 delivery contract, runbook.md
delivery-mode switching instructions.
-
-### Phase 2c/2e ? Backfill (planned, not started)
-### Phase 2c/2e ? Backfill engine + /backfill command ? DONE (PR #12)
-- **`backfill.py`**: `backfill_user()` scans all channel dialogs via
`client.iter_messages(offset_date=cutoff, wait_time=0.5s)` for the last `days` of videos. Reuses
`extract_video_meta` ? `videos` dedup table ? `FilterEngine` ? `deliver()` ? same pipeline as
the live watcher, so backfill + live = no duplicates. FloodWaitError caught and slept on.
Progress callbacks send status messages to Saved Messages.
-- **`/backfill` command**: `make_backfill_command_handler()` registered on the `me` chat in
`default_run_user`. Parses `/backfill [N][hdw]` (default 90 days, optional hour/week units).

```

```

Runs `backfill_user` in a background `asyncio.create_task` so the handler returns immediately.
Sends start/progress/completion messages to `me`.
-- **`manager.py`**: registers the command handler as a second `events.NewMessage(chats=["me"])`
in `default_run_user`, alongside the existing video watcher.
-- **Tests**: 127 passed @ 97.8% coverage. 15 new tests: backfill regex parsing (13 parametrized
including non-matching inputs), exports verification. `backfill_user` + command handler marked
`# pragma: no cover` (Telethon glue).
-- **`pyproject.toml`**: added per-file-ignore for RUF006 on backfill.py.
-- **Docs**: runbook.md updated with /backfill usage (h/w/d units).
-
-### Phase 2e ? Docs/runbook update ? DONE (PR #12, included)
-
-## v1.2 ? Per-user delivery target ? DONE
-
-### Phase 2f ? Configurable delivery destination (ADR 0005)
-- **`models.py`**: added `delivery_target: str = "me"` to `FilterConfig`. Accepts `me` (Saved
Messages, default), `@username`, or a numeric chat id string. A `field_validator` coerces int
chat ids to str for a uniform JSON blob. `min_length=1` rejects empty strings. Backwards-
compatible (old configs default to `me`).
-- **`delivery.py`**: renamed `forward_to_saved` ? `forward_to_target`, added `target: str =
"me"` to `forward_to_target`, `send_link_message`, and `deliver()`. The two hardcoded `me`
literals are gone ? delivery destination is now fully config-driven.
-- **`telethon_client.py`** + **`backfill.py`**: `make_new_message_handler`, `backfill_user`,
`make_backfill_command_handler`, `_run_backfill_and_report` all take `delivery_target` and
thread it to `deliver()`. **Backfill progress messages still go to `me`** (operational, not
curated content).
-- **`manager.py`**: passes `cfg.delivery_target` through to both the live handler and the
backfill command handler.
-- **Tests**: 146 passed @ 97.2% coverage; ruff clean. New tests: `deliver()` routes
forward/link/both to the configured target (default `me`, `@username`, numeric id), FakeClient
records target args, `delivery_target` JSON round-trip, old-config-without-field defaults to
`me`, int-coercion + empty-string rejection.
-- **Docs**: ADR 0005, design.md diagram + control-flow step 8, runbook.md "Routing matches to a
channel you own" section, README feature line.
-
-## v1.3 ? Content-based deduplication ? DONE
-
-### Phase 3a ? Two-tier content dedup + schema migration (ADR 0006)
-- **`models.py`**: added nullable `document_id: int | None` to `VideoRecord` + a
`content_fingerprint` property `(size_bytes, duration_s, height, width)` returning `None` if any
field is missing.
-- **`db.py`**: added `document_id` column, partial unique index `idx_videos_user_document`
(Tier 1, DB-enforced), and `get_video_by_document_id()` / `get_video_by_content_fingerprint()`
queries. **Introduced a `PRAGMA user_version` migration ladder**: migration 001 idempotently
adds the `document_id` column + index to legacy v1 databases. The base `_SCHEMA` no longer
creates the index (it references a column that may not exist on legacy DBs); the migration owns
it for both fresh and legacy DBs.
-- **`telethon_client.py`**: new `_extract_document_id()` helper; the live `on_new_message`
handler now runs three-tier dedup (Tier 0 instance ? Tier 1 document.id ? Tier 2 fingerprint)
cheapest-first.
-- **`backfill.py`**: same three-tier dedup applied to the historical backfill path.
-- **Tests**: 162 passed @ 97% coverage; ruff clean. New tests: fresh-DB column/index presence,
`user_version` bumped to latest, legacy-DB migration (column added, data preserved,
`document_id` defaults NULL), idempotent re-run, partial unique index enforces Tier-1 dedup
(rejects dup `document_id`, allows multiple NULLs).
-- **Docs**: ADR 0006, design.md data-model + control-flow step 4 + component diagram,
runbook.md dedup section.
-
-## Verification
1. **Unit tests green**: `pytest` ? filters, engine, db repos. Coverage ? 80% on `filters/` and
`db.py`.
2. **Lint clean**: `ruff check .` and `ruff format --check .` exit 0.
3. **Pre-commit passes** on a sample commit.
4. **Docker build**: `docker compose build` succeeds; container starts and logs "manager
started, N users".

```

-5. ****Dry-run ingestion**** (manual, against a test channel): post a short (<5s) and a long video to a channel the test account is in; confirm only the long one is forwarded to Saved Messages, and both rows exist in `videos` table with correct metadata.

-6. ****FloodWait path****: unit-test the delivery retry logic with a mocked `FloodWaitError(seconds=2)`.

-7. ****Restart resilience****: stop container, restart, confirm sessions still valid (no re-login prompt) and seen-state preserved.

-

-## Decisions / scope

-- ****Included in v1****: userbot ingestion (all subs), real-time forward to Saved Messages, 3 filters, multi-user, SQLite, Docker, docs, tests, pre-commit.

-- ****Excluded (v2)****: public-channel join (`/add @channel`), manual-add ingestion path, scheduled digest delivery, on-demand `/list` `/search`, ffmpegprobe `/inspect`, codec/keyword/mime filters.

-- ****Assumptions****: user provides their own `api\_id`/`api\_hash` from my.telegram.org; runs on a single host (no horizontal scaling needed at <100 users); acceptable to log in once and keep a persistent session file.

-

-## Further considerations

-1. ****Multi-user onboarding UX****: v1 bootstraps users via an interactive CLI script. Acceptable, or do you want a Telegram-based `/register` admin command? (Recommend: CLI for v1, command in v2.)

-2. ****Saved Messages vs. a dedicated bot-chat delivery target****: ? Resolved in v1.2 (ADR 0005) ? `delivery\_target` is now configurable per-user (`"me"`, `@username`, or chat id). Default remains Saved Messages.

-3. ****Filter match semantics****: AND (all rules must pass) vs. OR (any rule). (Recommend: AND by default ? "videos that satisfy ALL my criteria" ? which matches the stated use case of filtering TO a quality bar.)

+Prefer this file, [design.md](./design.md), and [runbook.md](./runbook.md) over

+older phase notes when checking current behavior.

42. assistant (2026-07-07T03:36:57.492Z)

The plan.md has one claim to verify: "migrations through `user\_version=5`". Let me check the actual migration version, and check the plan's F-phase issue numbers (#43-#49) ? those are harder to verify without GitHub but the PR numbers #50-#57 match the ADR. Let me verify the schema `user\_version`.

43. user (2026-07-07T03:36:59.862Z)

```
208:     then runs any pending migrations via the ``user_version`` ladder.
219:     await conn.execute("PRAGMA busy_timeout = 5000")
237:# Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
257:     """Register a migration that brings the DB up to ``user_version == version``."""
331:     """Run every registered migration whose version > current ``user_version``."""
332:     async with conn.execute("PRAGMA user_version") as cur:
340:         await conn.execute(f"PRAGMA user_version = {version}")
```

44. assistant (2026-07-07T03:37:02.477Z)

Let me find the highest registered migration version to verify the `user\_version=5` claim.

45. user (2026-07-07T03:37:04.677Z)

```
243:# 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
248:_MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
251:def _migration(
262:    _MIGRATIONS.append((version, fn))
336:    for version, fn in _MIGRATIONS:
350:@migration(1)
351:async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
367:@migration(2)
368:async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
519:@migration(3)
```

```
520:async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
682:@_migration(4)
683:async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
778:@_migration(5)
779:async def _migration_005_policy_decisions(conn: aiosqlite.Connection) -> None:
```

46. assistant (2026-07-07T03:37:09.456Z)

Migrations run through version 5. The plan.md claim "migrations through `user\_version=5`" is accurate.

Now let me run the test to verify the DB actually ends at user_version 5, and confirm save_filter_config merges (preserving policy keys). Let me verify save_filter_config behavior.

47. user (2026-07-07T03:37:11.167Z)

```
1902:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
1903-    """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1904-
1905-    ADR 0012: future pipeline keys live in the same ``policies.config_json``
1906-    object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1907-    merge the flat config fields into the existing object so unknown policy
1908-    keys survive legacy ``/set`` command updates.
1909-    """
1910-    existing = await get_default_policy(conn, user_id)
1911-    raw_config: dict[str, object] = {}
1912-    if existing is not None:
1913-        try:
1914-            raw_config = _policy_config_json_to_dict(existing.config_json)
1915-        except (json.JSONDecodeError, ValueError):
1916-            log.warning(
1917-                "resetting malformed policy config while saving FilterConfig user=%s "
1918-                "policy_id=%s",
1919-                user_id,
1920-                existing.id,
1921-                exc_info=True,
1922-            )
1923-            raw_config = {}
1924-    raw_config.update(cfg.model_dump(mode="json"))
1925-    await save_policy_config(conn, user_id, raw_config)
1926-
1927-
1928-# --- bulk helpers (used by the manager on startup) -----
1929-
1930-
1931-async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1932-    conn: aiosqlite.Connection, user_id: int, limit: int = 100
```

48. assistant (2026-07-07T03:37:18.464Z)

save_filter_config merges (preserving policy keys). ADR 0012 and design.md claims confirmed accurate.

Now let me read the runbook.md diff, which typically has the most operator-actionable claims (deploy steps, commands).

49. user (2026-07-07T03:37:19.969Z)

```
diff --git a/docs/runbook.md b/docs/runbook.md
index b2e3ad7..030c318 100644
--- a/docs/runbook.md
+++ b/docs/runbook.md
@@ -1,73 +1,116 @@
```

```

# Runbook: Telegram Video Filter Userbot

-Operational guide for deploying, onboarding users, and handling common issues.
-See [design.md](./design.md) for architecture and [plan.md](./plan.md) for
-build status.
+Operational guide for deployment, onboarding, source control, filtering,
+backfill, explainability, and troubleshooting.

## 1. Prerequisites

- A Telegram account you control.
-- `api_id` and `api_hash` from <https://my.telegram.org> ? *API development
- tools*. Create an app (any name/desc) to get these.
-- Docker + Docker Compose (recommended), **or** Python 3.11+.
-- A persistent directory mounted at `/data` (holds the SQLite DB + session
- files ? treat as a credential).
-- **Optional but recommended**: install `cryptg` (`pip install cryptg`) for
- native crypto acceleration. Without it Telethon falls back to slower
- pure-Python encryption (you'll see a warning at startup). For Docker, add
- `cryptg` to the image.
- > **_Note:_** `cryptg` is included as a project dependency by default
- > (`pyproject.toml`); you should already have it installed.
-

-## 2. First-time deploy (Docker)
+- `api_id` and `api_hash` from <https://my.telegram.org>.
+- Docker + Docker Compose, or Python 3.11+.
+- A persistent `data/` directory for SQLite and Telethon session files.
+- Optional local virtualenv for scripts/tests:
+
+```powershell
+python -m venv .venv
+.\venv\Scripts\python.exe -m pip install -e .[dev]
+```
+
+`cryptg` is a runtime dependency and should already be installed through the
+project package.
+
+## 2. First Deploy

```bash
1. clone
git clone https://github.com/avidullu/telegram-video-filter.git
cd telegram-video-filter

2. configure env
cp .env.example .env
edit .env: set TG_API_ID, TG_API_HASH, DB_PATH=/data/app.db

3. (optional) pre-create the data volume
mkdir -p data
edit .env: TG_API_ID, TG_API_HASH, DB_PATH, SESSIONS_DIR

4. build + run in background
docker compose up -d --build
docker compose logs -f app
```

-Expected startup log: `manager_started {users: 0}` until you onboard a user.
+Expected first log state: the manager starts with no runnable sessions until a
+user is bootstrapped.

-## 3. Onboard a user (interactive, once per user)
+## 3. Onboard a Telegram User Session

-v1 onboards via a host-side CLI that performs the interactive Telegram login and

```

-writes a session file into `data/sessions/`.
+Run the bootstrap command in the same environment that can write to `data/`:

```
```bash
-# from the repo root, inside the same env/container that runs the app
python -m tg_video_filter.bootstrap
-# ? prints the banner (session dir + db path)
-# ? prompts: phone (E.164), then the login code Telegram sends you,
-# then 2FA password if enabled.
-# ? creates data/sessions/user_{id}.session
-# ? inserts a users row (enabled=1) + default filter_config
```
```

-After onboarding, restart the container so the manager picks up the new user:

+It prompts for:

+
+1. phone number,
+2. Telegram login code,
+3. 2FA password if enabled.

+
+It then creates:

+
+- `accounts` row,
+- `telegram\_sessions` row,
+- legacy-compatible `users` view data,
+- default policy/config rows,
+- `data/sessions/user\_{id}.session`.

+
+Restart the manager after onboarding:

```
```bash
docker compose restart app
-# ? logs: client_connected {user_id: <id>}
```
```

-> ?? **The session file is a credential.** Anyone with `user\_{id}.session` can
-> act as that Telegram account. Keep `data/` private and backed up.

+Session files are credentials. Back up and protect `data/`.

+
+## 4. Live Source Scanning

+
+Live ingestion is opt-in per source.

+
+Unknown sources are lazily discovered as `watched=false`; they are listed but
+not ingested until you watch them. Send commands to your own Saved Messages:

+
+```text
+/channels
+/watch <id|@name|title-fragment>
+/unwatch <id|@name|title-fragment>
+/watchlist
+```

+
+Typical flow:

+
+1. Let new traffic arrive, or run `/backfill` to discover historical dialogs.
+2. Run `/channels`.
+3. Run `/watch -1001234567890` or `/watch Some Channel`.
+4. Confirm with `/watchlist`.

+
+## 5. Filter Commands

+
+These commands persist config changes and hot-reload the in-memory filter
+engine; no restart is needed for these fields.

+

```
+```text
+/show config
+/show commands
+/set size 10MB
+/set length 15m
+/set max length 15m
+/set min length 5s
+/set min resolution 720p
+/set min res 720p
+/set exclude gif on
+/set exclude round off
+/set exclude nosound on
+```
+
+Units:
```

```
-## 4. Adjusting filters
+- size: `KB`, `MB`, `GB`, `TB`; default is MB.
+- duration: `s`, `m`, `h`; default is seconds.
```

```
-Filters are stored per-user as JSON in `filter_config`. Default v1 config:
+Default flat filter config for fresh installs:
```

```
```json
{
@@ -75,224 +118,219 @@ Filters are stored per-user as JSON in `filter_config`. Default v1
config:
 "max_duration_s": 7200,
 "target_height": 1080,
 "max_size_mb": 2048,
+ "exclude_gif": true,
+ "exclude_round": true,
+ "exclude_nosound": false,
 "delivery_mode": "forward",
 "delivery_target": "me"
}
```
```

```
-To change a user's config (no v1 UI yet ? use sqlite3 directly):
+## 6. Delivery Mode and Target
```

```
-```bash
-sqlite3 data/app.db
-# find the user
-SELECT id, phone FROM users;
-# update (example: allow up to 4h, drop min duration)
-UPDATE filter_config
-  SET config_json = json('{"min_duration_s":0,"max_duration_s":14400,"target_height":1080,"max
_size_mb":2048,"delivery_mode":"forward"}')
- WHERE user_id = <id>;
-.quit
-docker compose restart app
-```
```

```
+Delivery mode and target are stored in policy/config JSON. There is no
+Saved-Messages command for changing them yet, so update deliberately and restart
+the manager.
```

```
-### Backfill historical videos (v1.1)
+Modes:
```

```
-> ?? Backfill scans all channels for historical videos. On large channels
-> it can run for minutes and may trigger Telegram rate limits. Progress is
-> reported in real time to your Saved Messages.
+| Mode | Behavior |
+|---|---|
```

```
+| `forward` | Forward the Telegram video message. |
+| `link` | Send a `t.me` link plus metadata summary; falls back to forward if no link exists. |
+| `both` | Forward and send the link summary. |
```

-While the manager is running, send a message to your ****Saved Messages****:
+Targets:

```
+| Target | Meaning |
+|---|---|
+| `me` | Saved Messages. |
+| `@your-feed` | Public channel or chat username. |
+| `-1001234567890` | Numeric Telegram chat/channel id. |
```

+Use the helper script for targets:

```
+`powershell
+.\.venv\Scripts\python.exe scripts\set_delivery_target.py <account_id> <target>
```
```

```
-/backfill # scan last 90 days (default)
-/backfill 7d # scan last 7 days
-/backfill 8h # scan last 8 hours
-/backfill 2w # scan last 2 weeks
-/backfill 30 # scan last 30 days
-```
```

-The bot responds:

+If Telethon cannot resolve a target, refresh the entity cache:

```
+`powershell
+.\.venv\Scripts\python.exe scripts\resolve_target.py <account_id> <target>
```
```

```
-? Starting backfill of all channels (last 90 days)...
-? Scanning #ChannelName (last 90 days)...
-  #ChannelName: 142 videos, 8 matched so far...
-? #ChannelName: 156 videos, 11 matched
-? Backfill complete: 5 channels scanned, 823 videos found, 47 matched
```

+Then restart:

```
+`bash
+docker compose restart app
```
```

```
-- Videos already seen (by live watcher or prior backfill) are **skipped**.
-- Matches are delivered according to your current `delivery_mode`.
-- If a channel triggers a flood wait, it sleeps and continues.
+Backfill progress messages always go to Saved Messages; delivered matches honor
+`delivery_target`.
```

-### Tuning filter criteria live (v1.3)  
+## 7. Backfill

-While the manager is running, send these commands to **\*\*Saved Messages\*\***:  
+Backfill scans historical channel/group dialogs and can be expensive on large  
+accounts. It does not use the live `watched` gate; run it intentionally.

```
-```
-/show config # display current filter settings
-/show commands # list all available commands
-/set size 10MB # set max file size
-/set length 15m # set max video duration
-/set max length 15m # same as above (alias)
-/set min length 5s # set min video duration
-/set exclude gif on # exclude Telegram GIFs
-/set exclude round on # exclude circular video messages
```

```

-/set exclude nosound on # exclude videos without audio
+```text
+/backfill
+/backfill 8h
+/backfill 7d
+/backfill 2w
+/stop
```

```

-All changes persist to the DB and take effect immediately ? no restart needed.
 -Exclusions are ****off by default**** for backwards compatibility.
 +Behavior:

```

-### Switching delivery mode (v1.1)
+- default window is 90 days;
+- dedup skips already-seen content;
+- matching content is delivered with current delivery mode/target;
+- progress is reported to Saved Messages;
+- flood waits are slept through and logged.

```

-The ``delivery\_mode`` field controls how matched videos are delivered to your
 -Saved Messages:
 +## 8. Policy Pipeline

```

-	Mode	What you get
-| ``forward`` (default) | The video itself is forwarded (re-uploaded). |
-| ``link`` | A text message with a ``t.me`` link + metadata summary (resolution, duration, size). No re-upload. |
-| ``both`` | Forward the video AND send the link message. |
+Metadata filters run first. If they pass, ADR 0012 policy stages run in order:

```

-Example ? switch to link-only mode:
 +1. topic preferences,
 +2. URL/domain reputation,
 +3. caption safety,
 +4. deep inspection boundary.

```

-```sql
-UPDATE filter_config
-  SET config_json = json_set(config_json, '$.delivery_mode', 'link')
-  WHERE user_id = <id>;
-```
+Flat `/set` commands update `FilterConfig` fields inside the default policy
+without erasing unknown policy keys.

```

-Then restart the manager. Old configs without ``delivery\_mode`` default to
 -``forward`` (backwards-compatible).
 +Policy-stage config currently has no friendly command UI. Edit
 +`policies.config\_json` carefully when needed. Example:

```

-### Routing matches to a channel you own (v1.2)
-
-By default matches go to your **Saved Messages**. Set ``delivery_target`` to
-route them to a channel instead (see ADR 0005):
+```sql
+UPDATE policies
+  SET config_json = json_set(
+    config_json,
+    '$.topics',
+    json('{"include":["ai"],"exclude":["crypto"],"mode":"any"}')
+  )
+  WHERE account_id = <account_id>
+  AND is_default = 1;
+```

```

```

-	Value	Meaning
-| ``"me"`` (default) | Saved Messages. |
-| ``"@your-feed"`` | A public channel the account is a member of. |
-| ``"-1001234567890"`` | A private channel's chat id (the ``-100?`` form). |
+Caption and domain checks:

```sql
-UPDATE filter_config
- SET config_json = json_set(config_json, '$.delivery_target', '@my-movie-feed')
- WHERE user_id = <id>;
+UPDATE policies
+ SET config_json = json_set(
+ config_json,
+ '$.security_checks',
+ json('{
+ "blocked_domains": ["bad.example"],
+ "unsafe_caption_terms": ["malware"],
+ "unsafe_caption_patterns": ["free\\\\s+money"]
+ }')
+)
+ WHERE account_id = <account_id>
+ AND is_default = 1;
```

-Then restart the manager. Requirements:
+Keep `deep_inspect` disabled unless a production fetcher/budget is explicitly
+wired:
+
+```json
+{ "deep_inspect": false }
+```

-- The userbot account must be a member of the target channel and have
- post rights (admin) if posting is restricted.
-- Backfill progress messages still go to Saved Messages ? only matched
- videos/links honor `delivery_target`.
-- Old configs without `delivery_target` default to ``"me"`` (backwards-compatible).
+## 9. Explain Decisions

-Instead of hand-writing SQL, use the helper script (run with the venv python,
-not the bare `python` on your PATH ? the project's deps live in .venv):
+Use /why <content_item_id> from Saved Messages.

- # Windows (PowerShell)
- .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
- # macOS / Linux
- .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
+Find recent content item ids:

-#### Troubleshooting: "Cannot find any entity corresponding to ..."
+```sql
+SELECT id, provider, provider_message_id, matched, seen_at
+ FROM content_items
+ ORDER BY id DESC
+ LIMIT 20;
+```

-If delivery fails with ``Cannot find any entity corresponding to "-100..."``,
-Telethon's session cache has never recorded that channel's `access_hash`.
-Resolve and cache it with::
+`/why` behavior:

- .\\.venv\\Scripts\\python.exe scripts\\resolve_target.py <user_id> <target>
+- reads only content owned by the requesting account;

```

- + renders persisted `policy\_decisions` traces when present;
- + falls back to recorded item status plus current metadata-filter replay when
- + no policy decision exists;
- + escapes untrusted captions/URLs/metadata and caps the reply under Telegram's
- + message limit.

- This stops the manager (the session file can't be shared), refreshes the
- entity cache via ``get\_dialogs()``, verifies the target resolves, and writes
- the cached entity back to the session file. Then restart the manager.
- +The fallback replay uses current config and persisted metadata. It can differ
- +from the original decision if config changed, and GIF/round/nosound flags are
- +not persisted for replay.

4b. How video deduplication works (v1.3)

10. Deduplication

- The userbot will not forward the same video twice thanks to a **three-tier**
- dedup** pipeline (checked cheapest-first; see ADR 0006):
- +Dedup is automatic and metadata-first:

| Tier | Key | Catches |
|------|---|--|
| 0 | (user_id, channel_id, message_id) | The same message instance (re-edits, restarts). |
| 1 | (user_id, document_id) | The same Telegram file forwarded across channels ? Telegram reuses `document.id`. DB-enforced via a partial unique index. |
| 2 | (size_bytes, duration_s, height, width) | A re-upload of the identical file (new `document.id`, same bytes). Heuristic lookup, no DB constraint. |

- This is fully automatic and adds no bandwidth cost ? it relies only on metadata
- already present in the message. A re-encode (different resolution/codecs/size)
- is **intentionally not deduped** ? it is a different file and is evaluated
- independently by the filter engine.
- +| 0 | source/message | Same Telegram message instance. |
- +| 1 | Telegram `document.id` | Same file forwarded across channels. |
- +| 2 | size/duration/height/width | Re-upload of identical media metadata. |

- Database migration**: existing v1 databases are upgraded automatically on the
- next `init\_db()` call via a `PRAGMA user\_version` migration ladder. The
- `document\_id` column is added with `ALTER TABLE` (nullable; existing rows get
- `NULL`) and the partial unique index is created. The migration is idempotent
- and safe to re-run. No manual SQL is needed.
- +Re-encodes are intentionally treated as distinct content.

5. Rotate a user session

11. Backups

- Use when a session is suspected compromised, a phone number changes, or login
- starts failing.
- +The important state is `data/`:

```

```bash
docker compose stop app
rm data/sessions/user_{id}.session # remove old session
also kill it from Telegram side: Settings ? Devices ? terminate the session
python -m tg_video_filter.bootstrap # re-login interactively
tar -czf backup-$(date +%F).tar.gz data/
docker compose start app
```

```

6. Handle `FloodWaitError`

- Telegram rate-limits automated actions. The delivery layer already:
- 1. catches `FloodWaitError`,
- 2. sleeps `e.seconds + small\_jitter`,

```

-3. retries **once**.
-
-If you still see floods frequently:
-- Reduce how many channels the account is in, or
-- Temporarily disable the user (`UPDATE users SET enabled=0 WHERE id=?`) and
- wait, or
-- Tighten filters so fewer videos are forwarded.
+Restore by stopping the app, replacing `data/`, and starting again.

-Persistent abuse-level automation can get the **account** restricted ? this is
-the user's responsibility (see ADR 0001).
+## 12. Rotate a Session

-## 7. Backups
-
-The entire state is the `data/` directory:
+Use this when a session is compromised or login starts failing:

```bash
docker compose stop app
tar -czf backup-$(date +%F).tar.gz data/
rm data/sessions/user_{id}.session
+# Also terminate the session from Telegram: Settings -> Devices.
+python -m tg_video_filter.bootstrap
docker compose start app
```

-Restore = stop, replace `data/`, start.
+## 13. Observability

-## 8. Observability
-
-v1 logs JSON Lines to stdout. Tail with:
+Logs go to stdout:

```bash
docker compose logs -f app | jq .
docker compose logs -f app
```

-Key events: `manager_started`, `client_connected`, `video_seen`
-(`{matched, reasons}`), `video_forwarded`, `flood_wait {seconds}`,
-`client_disconnected`.
+Useful events:
+
+- `manager_started`
+- `client_connected`
+- `video_seen`
+- `video_dedup_document_id`
+- `video_dedup_fingerprint`
+- `flood_wait`
+- `config_updated`
+- `client_disconnected`

-There is no v1 metrics endpoint; rely on logs + Docker restart policy
-(`restart: unless-stopped` in compose).
+There is no metrics endpoint yet.

-## 9. Troubleshooting
+## 14. Troubleshooting

Symptom	Likely cause	Fix	
-	`client_disconnected` immediately on start	session file missing/invalid	re-run bootstrap ($5)

```

```

-| No `video_seen` events at all | account not in any channel posting videos, or filters too
strict | post a test video to a channel; relax filters ($4) |
-| Many `flood_wait` logs | forwarding too fast | see $6 |
-| `videos.matched=1` but `forwarded=0` | forward failed after retry | check logs for the error;
the row will not auto-retry ? manually forward or re-trigger by deleting the row |
-| Container restarts loop | DB locked / bad env | `docker compose logs app`; verify `.env` and
that `data/` is writable |
+| Manager starts with zero users | No bootstrapped session | Run `python -m
tg_video_filter.bootstrap`, then restart. |
+| No live `video_seen` events | Source is not watched | Run `/channels`, then `/watch
<source>`. |
+| `/channels` is empty | No source discovered yet | Wait for new traffic or run `/backfill`
carefully. |
+| Delivery target cannot resolve | Telethon session cache lacks entity | Run
`scripts/resolve_target.py`, then restart. |
+| Lots of `flood_wait` logs | Too much delivery/backfill traffic | Stop backfill, reduce
watched sources, tighten filters. |
+| `/why` says no trace | Item predates F6 or was metadata-filtered before policy stages | Use
fallback output; future decisions should have traces. |
+| DB locked/container loop | Shared process or bad volume permissions | Stop duplicate
processes; check `data/` ownership and `.env`. |

```

50. assistant (2026-07-07T03:37:24.977Z)

The runbook has several concrete claims to verify:

1. Helper scripts `scripts/set\_delivery\_target.py` and `scripts/resolve\_target.py` ? do they exist and take account_id?
2. Log events `video\_dedup\_document\_id` and `video\_dedup\_fingerprint` ? do they exist?
3. `/set length 15m` ? does the `m` suffix mean minutes for duration? And "duration: `s`, `m`, `h`; default is seconds."
4. Filter commands "hot-reload the in-memory filter engine; no restart is needed" ? is that true?
5. `/watch Some Channel` example.

Let me verify these.

51. user (2026-07-07T03:37:28.569Z)

```

resolve_target.py
set_delivery_target.py
---dedup events---
Binary file src/tg_video_filter/__pycache__/backfill.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/telethon_client.cpython-313.pyc matches
dedup_document
dedup_fingerprint
video_dedup
video_dedup_document_id
video_dedup_fingerprint
---set length regex---
33:_LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
34:_MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
145:         "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
146:         "    or `/set max length <N><unit>` ? same as above",
148:         "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
5s`)",
348:         # --- /set length <value><unit> -----
349:         m = _LENGTH_RE.match(text)
355:         return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
368:         # --- /set min length <value><unit> -----
369:         m = _MIN_LENGTH_RE.match(text)
375:         return CommandResult("? Invalid duration format. Use e.g. `/set min length
5s`.")

```

52. assistant (2026-07-07T03:37:33.535Z)

Scripts exist, dedup log events (`video\_dedup\_document\_id`, `video\_dedup\_fingerprint`) exist, and the length regex accepts `[smh]` units. Let me verify the scripts take account_id as first arg (runbook says ``), and check the size units and that "hot-reload no restart" claim.

53. user (2026-07-07T03:37:36.415Z)

```
=== set_delivery_target usage ===
6:  .\\.venv\\Scripts\\python.exe scripts\\set_delivery_target.py <user_id> <target>
8:  .venv/bin/python scripts/set_delivery_target.py <user_id> <target>
31:  if len(sys.argv) != 3:
32:      print("Usage: python scripts/set_delivery_target.py <user_id> <target>",
file=sys.stderr)
36:  user_id = int(sys.argv[1])
37:  target = sys.argv[2].strip()
50:      "WHERE user_id = ?",
51:      (target, user_id),
55:      print(f"ERROR: no filter_config row for user_id={user_id}.", file=sys.stderr)
64:      "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
=== resolve_target usage ===
15:  .\\.venv\\Scripts\\python.exe scripts\\resolve_target.py <user_id> <target>
17:  .venv/bin/python scripts/resolve_target.py <user_id> <target>
42:async def _resolve(user_id: int, target: str) -> int:
49:  session_path = Path(settings.sessions_dir) / f"user_{user_id}"
91:  if len(sys.argv) != 3:
92:      print("Usage: python scripts/resolve_target.py <user_id> <target>", file=sys.stderr)
95:  return asyncio.run(_resolve(int(sys.argv[1]), sys.argv[2].strip()))
=== size units ===
32:_SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmg]t?b?)$", re.IGNORECASE)
107:     return "0MB"
109:     gb = mb / 1024
110:     if gb == int(gb):
111:         return f"{int(gb)}GB"
112:     return f"{gb:.1f}GB"
113: return f"{mb}MB"
143:     "/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
144:     "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing
`B`",
330:     m = _SIZE_RE.match(text)
```

54. assistant (2026-07-07T03:37:42.803Z)

Two important findings emerge:

1. **The helper scripts take ``<user_id>``, not ``<account_id>``.** The runbook (and design docs) now use ``<account_id>`` in the invocation examples: ``scripts/set_delivery_target.py <account_id> <target>`` and ``scripts/resolve_target.py <account_id> <target>``, and even the `/why` SQL uses `WHERE account_id = <account_id>`. But the scripts themselves query `WHERE user_id = ?` against the legacy `filter_config` table. Since legacy `users`/`accounts` share the same id via shims, this may or may not be a real mismatch. Let me verify whether `account_id == user_id` (the shim keeps ids identical) ? if so, this is cosmetic; if not, it's a high-severity operator break.

2. **The size units in runbook say `KB`, `MB`, `GB`, `TB`** and README says the same (`KB`, `MB`, `GB`, or `TB`). But the actual regex is `[kmg]t?b?` so it accepts `t`/`TB``. Let me check whether TB is actually handled in the parsing logic (the regex matches `t`` but does the code compute TB?).

Let me verify both.

55. user (2026-07-07T03:37:44.158Z)

```
330         m = _SIZE_RE.match(text)
```

```

331         if m:
332             value_str, unit = m.group(1), m.group(2) or ""
333             # NOTE: ValueError is unreachable here ? the regex already validates
334             # the unit against [kmgT]?b?, so parse_size_mb can never raise.
335             # Kept for defensive parity with the original commands.py refactor.
336             try:
337                 new_mb = parse_size_mb(value_str, unit)
338             except ValueError:
339                 return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
340             if new_mb < 1:
341                 return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
342             cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
343             return CommandResult(
344                 f"? Max file size set to
**[_format_size(new_mb)]**.\n\n{format_config(cfg)}",
345                 config_changed=True,
346             )
347

```

56. assistant (2026-07-07T03:37:49.250Z)

The regex accepts `t` (TB), so the runbook/README listing TB as a supported unit is accurate as an input, though the in-app help text (`format\_commands`) only advertises MB/GB/KB. The docs listing TB is not wrong (it works). Let me confirm parse_size_mb handles `t`.

Now the more important question: the `` vs `` script arg mismatch, and whether account_id equals user_id. Let me check the shim relationship.

57. user (2026-07-07T03:37:50.960Z)

```

57: def parse_size_mb(value_str: str, unit: str) -> int:
58-     """Parse a size value + unit string into mebibytes."""
59-     value = int(value_str)
60-     unit = unit.lower().rstrip("b")
61-     multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
62-     if unit in multipliers:
63-         raw = value * multipliers[unit]
64-         if unit == "k":
65-             raw = max(1, raw // 1024)
66-         return raw
67-     raise ValueError(f"Unknown size unit: {unit}")
68-
69-
70: def parse_duration_s(value_str: str, unit: str) -> int:
71-     """Parse a duration value + unit string into seconds."""
72-     value = int(value_str)

```

58. user (2026-07-07T03:37:52.701Z)

```

43: CREATE TABLE IF NOT EXISTS accounts (
52:     account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
62:     user_id       INTEGER NOT NULL REFERENCES accounts(id),
66:     UNIQUE (user_id, channel_id)
71:     user_id       INTEGER NOT NULL REFERENCES accounts(id),
83:     UNIQUE (user_id, channel_id, message_id)
87:     user_id       INTEGER PRIMARY KEY REFERENCES accounts(id),
99:     account_id    INTEGER NOT NULL REFERENCES accounts(id),
107:    UNIQUE (account_id, provider, provider_source_id)
112:    account_id     INTEGER NOT NULL REFERENCES accounts(id),
129:    UNIQUE (account_id, source_id, provider_message_id),
130:    UNIQUE (account_id, dedup_key)
135:    account_id     INTEGER NOT NULL REFERENCES accounts(id),

```

```

141:    UNIQUE (account_id, name)
146:ON policies(account_id) WHERE is_default = 1;
150:    account_id INTEGER NOT NULL REFERENCES accounts(id),
157:    UNIQUE (account_id, provider, target)
162:    account_id INTEGER NOT NULL REFERENCES accounts(id),
169:    UNIQUE (account_id, source_id, destination_id)
363:    "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"

```

[Showing results with pagination = limit: 20]

59. assistant (2026-07-07T03:38:02.710Z)

Key finding confirmed: ``filter_config.user_id`` is a PRIMARY KEY that ``REFERENCES accounts(id)``. So ``user_id`` in the legacy ``filter_config`` table IS the same integer as ``accounts.id`` (`account_id`). The command handlers even pass ``user_id`` where the DB layer treats it as `account_id` (e.g., ``set_source_watched(conn, user_id, ...)`` uses ``account_id`` column). So the runbook's use of ``<account_id>`` as the script argument resolves to the same integer ? it is NOT a functional break. The script's own usage string says ``<user_id>`` but they're the same id. This is a cosmetic/terminology inconsistency, not an operator-breaking error. I'll note it as low at most, but since the value is identical it's arguably not even misleading in outcome. I'll not flag it as a hard error.

Also confirmed: TB size unit (``t``) works.

Now let me check a few remaining runbook claims:

1. ``/set exclude round off`` example and "Exclusions are off by default" was REMOVED ? good, the new doc correctly says GIF/round excluded, nosound allowed. Let me verify the runbook's default JSON block: ``exclude_gif: true, exclude_round: true, exclude_nosound: false``. That matches ground truth. Good.
2. The ``/set length 15m`` ? `parse_duration_s` with ``m``. Let me confirm ``m`` = minutes (runbook says "duration: s, m, h").
3. The design.md claim about ``/why`` breaking raw autolinks. Let me verify the filter hot-reload claim and duration units.

60. user (2026-07-07T03:38:03.702Z)

```

70     def parse_duration_s(value_str: str, unit: str) -> int:
71         """Parse a duration value + unit string into seconds."""
72         value = int(value_str)
73         unit = unit.lower()
74         multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
75         if unit in multipliers:
76             return value * multipliers[unit]
77         raise ValueError(f"Unknown time unit: {unit}")
78
79
80     # --- formatting helpers -----
81
82
83     def _format_duration(seconds: int | None) -> str:
84         if seconds is None:

```

61. assistant (2026-07-07T03:38:07.993Z)

Duration units s/m/h confirmed (default seconds). Now let me verify the "hot-reload the in-memory filter engine; no restart is needed" claim in the runbook §5, and the design.md ``/why`` claim about "breaks raw autolinks". Let me check how `config_changed` propagates to a live engine reload, and the `/why` escaping.

62. user (2026-07-07T03:38:09.892Z)

No matches found

63. user (2026-07-07T03:38:14.183Z)

```
src/tg_video_filter/manager.py:143:         engine_holder: dict[str, FilterEngine] = {ENGINE_KEY:
FilterEngine(cfg)}
src/tg_video_filter/command_service.py:254:     replay =
FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
src/tg_video_filter/command_service.py:289:     and ``config_changed`` to decide whether to hot-
reload the filter engine.
src/tg_video_filter/command_service.py:293:     config_changed: bool = False
src/tg_video_filter/command_service.py:312:         if result.config_changed:
src/tg_video_filter/command_service.py:313:             engine_holder["engine"] =
FilterEngine(current_cfg)
src/tg_video_filter/command_service.py:345:                 config_changed=True,
src/tg_video_filter/command_service.py:365:                 config_changed=True,
src/tg_video_filter/command_service.py:385:                 config_changed=True,
src/tg_video_filter/command_service.py:397:                 config_changed=True,
src/tg_video_filter/command_service.py:421:                 config_changed=True,
src/tg_video_filter/command_service.py:608:     "config_updated user=%s max_size_mb=%s
min_duration_s=%s max_duration_s=%s "
```

64. assistant (2026-07-07T03:38:18.561Z)

Hot-reload of the filter engine on config change is confirmed (line 312-313: ``engine_holder["engine"] = FilterEngine(current_cfg)``). The runbook §5 "hot-reload the in-memory filter engine; no restart is needed" is accurate.

Now let me verify the design.md ``/why`` claim: "breaks raw autolinks, and caps replies below Telegram's message limit." And check the runbook §9 fallback claim "GIF/round/nosound flags are not persisted for replay." Let me look at the `/why` handler and the `content_item_to_video_meta`.

65. user (2026-07-07T03:38:19.869Z)

```
async def _handle_why(
    self,
    conn: aiomysqlite.Connection,
    user_id: int,
    item_id_text: str | None,
    current_cfg: FilterConfig,
) -> CommandResult:
    """Explain a recorded content item's policy decision."""
    if item_id_text is None or not item_id_text.strip().isdecimal():
        return CommandResult("? Usage: `/why <content_item_id>`")
    item_id = int(item_id_text.strip())
    if item_id < 1:
        return CommandResult("? Usage: `/why <content_item_id>`")

    item = await db.get_content_item_for_account(conn, user_id, item_id)
    if item is None:
        return CommandResult(f"? Content item `{item_id}` was not found for your account.")

    assert item.id is not None
    decisions = await db.list_policy_decisions_for_content(conn, item.id)
    if decisions:
        return CommandResult(_format_policy_decisions(item, decisions))
    return CommandResult(_format_missing_policy_decision(item, current_cfg))

async def _resolve_source(
    self, conn: aiomysqlite.Connection, user_id: int, target: str
) -> Source | None:
    """Resolve a user-provided source identifier to a ``Source`` row.

    Accepts numeric channel id, ``@username``, or channel title.
    ``@`` prefix is stripped before lookup.
    """
    target = target.removeprefix("@")
```

```

# Try by provider-native numeric id (the common case).
src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
if src is not None:
    return src
# Try by @username.
src = await db.get_source_by_provider_id(conn, user_id, "telegram", f"@{target}")
if src is not None:
    return src
# Try by title (case-insensitive substring match).
all_srcs = await db.list_account_sources(conn, user_id)
target_lower = target.lower()
for s in all_srcs:
    if s.title and target_lower in s.title.lower():
        return s
return None

```

```

# -- internal -----

```

```

async def _update(
    self,
    conn:aiosqlite.Connection,
    user_id: int,
    current: FilterConfig,
    max_size_mb: int | None = None,
    max_duration_s: int | None = None,
    min_duration_s: int | None = None,
    target_height: int | None = None,
    exclude_gif: bool | None = None,
    exclude_round: bool | None = None,
    exclude_nosound: bool | None = None,
) -> FilterConfig:
    """Apply partial updates to the account's config, persist, and return."""
    updates: dict[str, object] = {}
    if max_size_mb is not None:
        updates["max_size_mb"] = max_size_mb
    if max_duration_s is not None:
        updates["max_duration_s"] = max_duration_s
    if min_duration_s is not None:
        updates["min_duration_s"] = min_duration_s
    if target_height is not None:
        updates["target_height"] = target_height
    if exclude_gif is not None:
        updates["exclude_gif"] = exclude_gif

```

66. assistant (2026-07-07T03:38:22.828Z)

`/why` is account-scoped via `get\_content\_item\_for\_account` ? confirmed. Let me verify the escaping/autolink-break and the `content\_item\_to\_video\_meta` replay (which doesn't have GIF/round/nosound flags, supporting runbook §9's caveat).

67. user (2026-07-07T03:38:26.231Z)

```

57: def parse_size_mb(value_str: str, unit: str) -> int:
70: def parse_duration_s(value_str: str, unit: str) -> int:
83: def _format_duration(seconds: int | None) -> str:
88:     if seconds < 60:
90:         if seconds < 3600:
103: def _format_size(mb: int | None) -> str:
108:     if mb >= 1024:
116: def format_config(cfg: FilterConfig) -> str:
117:     """Format a `FilterConfig` as a readable message (Telegram markdown)."""
135: def format_commands() -> str:
143:     """`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
145:     """`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
146:     """    or `/set max length <N><unit>` ? same as above",

```

```

148:         ``/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
5s`)",
150:         ``/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
151:         "    or `/set min res <N>p` ? same as above",
165:         ``/watch <id|@name>` ? Start watching a source for new videos",
166:         ``/unwatch <id|@name>` ? Stop watching a source",
170:         ``/why <content_item_id>` ? Explain why an item passed or failed",
178: def _truncate_text(text: str, max_len: int) -> str:
179:     if len(text) <= max_len:
184: def _safe_markdown_text(text: str, max_len: int) -> str:
190: def _format_json_value(value: object, max_len: int = 500) -> str:
192:     return _safe_markdown_text(text, max_len)
195: def _format_reply(lines: list[str]) -> str:
197:     if len(text) <= _REPLY_CHAR_LIMIT:
203: def _content_item_to_video_meta(item: ContentItem) -> VideoMeta:
214: def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) -> str:
221:     f"_{safe_markdown_text(f'{item.provider}:{item.provider_message_id}', 120)}",
224:     lines.append(f"Link: `{safe_markdown_text(item.link_url, 300)}`")

```

68. assistant (2026-07-07T03:38:30.182Z)

The `/why` output uses `\_safe\_markdown\_text`, `\_REPLY\_CHAR\_LIMIT`, and `\_content\_item\_to\_video\_meta` (which builds VideoMeta from stored metadata ? no GIF/round/nosound flags since those aren't stored on content_items). The design.md and runbook claims about markdown escaping, message-limit capping, and the replay caveat are all accurate.

Let me verify the "breaks raw autolinks" claim specifically in `\_safe\_markdown\_text`.

69. user (2026-07-07T03:38:35.376Z)

```

178     def _truncate_text(text: str, max_len: int) -> str:
179         if len(text) <= max_len:
180             return text
181         return f"{text[: max_len - 3]}..."
182
183
184     def _safe_markdown_text(text: str, max_len: int) -> str:
185         text = _truncate_text(text, max_len)
186         text = _AUTOLINK_RE.sub(r"\1: //", text)
187         return _MARKDOWN_ESCAPE_RE.sub(r"\\1", text)
188
189
190     def _format_json_value(value: object, max_len: int = 500) -> str:
191         text = json.dumps(value, default=str, ensure_ascii=False, sort_keys=True)

```

70. assistant (2026-07-07T03:38:45.801Z)

Confirmed: `\_AUTOLINK\_RE` breaks autolinks and `\_MARKDOWN\_ESCAPE\_RE` escapes markdown. All the `/why` design/runbook/README claims are accurate.

Now let me double-check one more concrete claim in the ADR 0009 note and README/design: the note says `/backfill` "can also discover historical dialogs" and `/channels` "lists discovered source rows." Also design §5 says "A first live message from an unknown Telegram channel/group creates a `sources` row with `watched=false`." That matches `should\_ingest\_source`.

One more: the ADR 0009 status note says routing_rules "exists, but the runtime still delivers through the account's current `delivery\_target` unless future routing logic is added." Let me quickly confirm routing_rules isn't wired into delivery. And verify the plan.md F-phase issue numbers claim isn't verifiable but PRs are consistent. Let me check routing usage.

71. user (2026-07-07T03:38:47.935Z)

---exit---

72. assistant (2026-07-07T03:38:52.744Z)

Confirmed: `routing\_rules` is not referenced in delivery/ingestion/manager ? it's a dormant skeleton, exactly as ADR 0009 and design.md (`routing\_rules` | Source -> policy -> destination mapping skeleton`) describe. Accurate.

Now let me reconsider the one substantive concern I found: the `delivery\_record\_failed` log event in design.md §10 that doesn't exist in code. Let me also double check whether the README's Development section claims are accurate (80% coverage gate). Let me verify the coverage floor.

73. user (2026-07-07T03:38:54.179Z)

```
pyproject.toml:113:fail_under = 80
```

74. assistant (2026-07-07T03:38:58.686Z)

Coverage gate of 80% confirmed (README and plan.md accurate).

Let me do a final check on one thing: the design.md observability list and the runbook observability list differ. The runbook lists `video\_dedup\_document\_id`, `video\_dedup\_fingerprint`, `config\_updated` (all real). The design.md §10 lists `delivery\_record\_failed` (not real). Let me confirm the actual event name in telethon_client for a failed delivery record, to characterize the design.md error precisely.

75. user (2026-07-07T03:39:01.029Z)

```
266:         # Lazily record the channel (best-effort; failures are non-fatal).
342:         await _record_policy_decision(
466:async def _record_policy_decision(
520:     "_record_policy_decision",
```

76. user (2026-07-07T03:39:04.960Z)

(Bash completed with no output)

77. user (2026-07-07T03:39:08.187Z)

```
221:             log.info(
249:             log.info(
275:             log.warning(
326:             log.info(
334:         log.info(
```

78. user (2026-07-07T03:39:12.818Z)

```
        log.info(
            "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
        log.info(
            "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
        log.warning(
            "failed to upsert channel %s for user %s", channel_id, user.id, exc_info=True
        log.info(
            "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
        log.info(
            "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
```

79. assistant (2026-07-07T03:39:23.936Z)

Confirmed. The actual events include `video\_dedup\_document\_id`, `video\_dedup\_fingerprint`, `video\_dedup\_race`, `video\_seen`. There is no `delivery\_record\_failed` anywhere in the codebase. The design.md §10 observability list introduces `delivery\_record\_failed` as a supposed "important event" ? it does not exist. The actual delivery-failure log lines are `delivery\_failed` / `delivery\_retry\_failed` (in delivery.py). This is a stale/fabricated event name in a newly written list, so an operator grepping logs for `delivery\_record\_failed` would

find nothing. Low-to-medium severity (log names are illustrative but this one is flat wrong and operator-facing in a troubleshooting context).

I've now verified all the major claims. Let me summarize my findings. The vast majority of the doc refresh is accurate and matches merged code. Findings:

1. **design.md §10** lists ``delivery_record_failed`` as an important log event ? no such event exists in code (actual: ``delivery_failed`/`delivery_retry_failed``). Low severity.

Let me do one last spot-check on the README "What Is Not Built" claim that could mislead: "A production deep-inspection fetcher/downloader" is listed as not built ? matches ground truth (ctx.fetcher is None). And ADR 0012 status note doesn't overstate. Good.

Let me verify there are no other fabricated log events in design §10 (``config_updated``, ``flood_wait`` ? already confirmed real; ``client_disconnected`` real). Only ``delivery_record_failed`` is bogus.

One more: the README/design/runbook say ``/set exclude gif|round|nosound on|off``. Confirmed real. And README quick-start ``docker compose up -d --build`` then bootstrap then restart ? matches runbook.

Let me finalize.

80. user (2026-07-07T03:39:30.314Z)

Structured output provided successfully